ARC-SPRUCE: Post-Quantum Blind Signatures and Anonymous Rate-Limited Credentials from Lattices

ACM Reference Format:
. 2026. ARC-SPRUCE: Post-Quantum Blind Signatures and Anonymous Rate-Limited Credentials from Lattices. In . ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

0.1 TODO

- Add hardness assumption for Ajtai's commitment scheme, including formal parameters.
- Fully define security assumptions for ISIS-f and that h is hiding μ, including formal parameters.
- Fix section on BBS-type hash etc in preliminaries, as this is currently incomprehensible.

1 Introduction

Anonymous credentials (AC) Carsten: todo: add citations are cryptographic protocols that allow parties to convey certified information about themselves in a privacy-preserving way. There are usually three roles in ACs, namely *issuers*, *holders* and *verifiers*. More concretely, an AC scheme allows a holder to prove to a verifier that it possesses certain attributes, which have been authenticated by the issuer. Here, the authenticated attributes are usually called a *credential*, which is kept private by the user. Using a so-called *showing* process, the user can generate a presentation which convinces the verifier that attributes stored in its credential (or credentials) satisfy predicates without revealing any additional information. While there are numerous security properties that ACs can achieve, the most fundamental ones are unforgeability and unlinkability. Here, the former ensures that an issuer indeed generated a credential that a user presents, and that colluding issuers and verifiers may not link a presentation to a specific issuance session. Here, the latter should also ensure that multiple presentations cannot be traced to the same credential.

Certain AC use-cases [YW25, SK25] require that the credential is only valid for a *bounded* number of presentations in a given context (e.g. per epoch or per service) – without revealing which presentations come from the same holder up to the enforced limit. A key advantage of existing, pre-quantum ACs is their performance, e.g. with constant credential and presentation size, regardless of the enforced rate limit. However, the situation looks very different in the post-quantum setting. There, practically efficient AC schemes [BLNS23a, AGJ+24, PWFW23], are yet mostly absent. This is highly unfortunate, given the uptake of (privacy-preserving) digital identification schemes in e.g. the EU Digital Identity Wallet, and simultaneous requirement for deployed schemes to eventually be quantum-secure: given current migration timelines, post-quantum ACs may become mandatory in the next 5-10 years.

Building ACs. To understand how post-quantum ACs (with rate-limiting) are usually constructed, it is instructive to understand how classically-secure constructions work. Anonymous credentials are often built on top of a Blind Signature design. Here, one of the most well-known approach (which is also the most promising in the post-quantum setting) is due to Fischlin [Fis06]:

(1) The issuer initially creates a key pair $(\mathsf{sk}, \mathsf{vk})$ using an EUF CMA-secure signature scheme. We assume that user and verifier have vk, while the issuer keeps sk.
(2) The holder first commits to its message μ as t, using randomness r. It then sends t to the issuer.
(3) Upon receiving t, the issuer signs it and thus creates a signature sig. It then sends sig to the user, who checks its verifiability with vk.
(4) To create the blind signature, the user creates a Non-Interactive Zero-Knowledge Proof of Knowledge (NIZK) showing that it knows sig, r such that

$$\mathsf{Verify}_{\mathsf{vk}}(t, \mathsf{sig}) = 1 \wedge t = \mathsf{Commit}(\mu; r)$$

Combining existing post-quantum primitives generically, i.e. without further considerations, leads to non-competitive performance. Thus, existing works Carsten: todo: cite carefully choose commitment schemes, signatures and NIZKs such that the NIZK for the given relation has good proof size and prover/verifier time. However, the state-of-the-art is generally not considered efficient enough for practical deployments.

1.1 Contributions

In this work, we narrow the efficiency gap for post-quantum blind signatures and ACs substantially. For Blind Signatures, we achieve this by optimizing existing design ideas carefully and employ novel primitives. We then extend our construction to yield the first post-quantum rate-limiting anonymous credentials. Crucially, the size of our credential as well as the presentation is essentially independent of the rate limit.

We construct a Blind Signature based on a signature whose security relies on lattice assumptions together with a hash-based NIZK. We then extend this to an *Anonymous Rate-Limited Credential* (ARC) [YW25] by including a Pseudorandom Function (PRF) into our design. Using a key committed in the message that is signed by the issuer, the user is able to evaluate a PRF on the key on different nonces. This PRF output, which we call a *tag*, can be made public. Since the signature and thus the key remain hidden due to the use of a NIZK, this intuitively makes the scheme unlinkable. By including a proof of correct PRF evaluation into the NIZK, the user also cannot generate a differing tag for the same nonce. Therefore, any two presentations generated for the same credential are

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

linkable as they must have the same tag, which makes the scheme rate-limiting.

To design the Blind Signature, we follow the recent proposal of [BLNS23b, LSS24, DKLW25a] which also presents blind signatures and anonymous credentials based on lattice assumptions. We first let the holder commit to its message μ using an Ajtai commitment. Then, in an interactive process, issuer and signer will sample randomness which together with the committed message becomes a target $\mathbf{t}$ that will be signed by the issuer. Here, the user proves in zero knowledge that $\mathbf{t}$ is consistent with the committed message and the issuer-supplied randomness. The signature scheme is formed by a matrix $\mathbf{A}$ together with a trapdoor td that allows sampling of short preimages mod$q$ using a GPV-style [GPV08] preimage sampler. The issuer then uses td to sample a short preimage $\mathbf{u}$ such that $\mathbf{Au} = \mathbf{t} \bmod q$, which serves as the signature. To prove knowledge of a valid $\mathbf{u}$ we then use the SmallWood NIZK scheme [FR25]. While this approach makes the round complexity of issuance non-optimal (requiring 3 rounds instead of 1 round), it allows tighter parameters for the preimage sampler (as $\mathbf{t}$ is uniform) while making the statement proven in the NIZK structurally simple. To turn this construction into an ARC, we let the user additionally commit to a PRF key $\mathbf{k}$ for the Poseidon2 hash function [GKS23]. It turns out that the parameters such as the modulus q of Poseidon2 can be chosen to be compatible with those of the lattice commitment, signature scheme and NIZK, making the proof of PRF evaluation highly efficient. The holder can then attach an evaluation of the PRF on a nonce where the committed key is used, together with a proof of correct PRF evaluation, as part of the presentation. If the holder reuses the nonce, this will be noticed by a verifier keeping track of previously issued tags.

1.2 Technical overview

This work constructs ARC around a single protocol blueprint with message $\mathbf{m} = \mu \| \mathbf{k}$, where μ is a vector of attributes and $\mathbf{k}$ is a *secret* PRF key.

Issuance. Let $\mathbf{A}_{\mathsf{Com}}$ be a uniformly random and compressing matrix to be used for the Ajtai commitment. In the issuance protocol, the holder additionally samples short $\mathbf{r}_0^{\mathcal{H}}, \mathbf{r}_1^{\mathcal{H}}, \mathbf{r}$ to compute the commitment $\mathbf{c} = \mathbf{A}_{\mathsf{Com}}[\mu \| \mathbf{r}]$ that is sent to the issuer. It, in response, samples short $\mathbf{r}_0^{\mathcal{I}}, \mathbf{r}_1^{\mathcal{I}}$ that are returned to the holder. The target $\mathbf{t}$ is then computed as

$$\mathbf{t} := \mathit{hash}(\mu, \mathbf{k}, \mathsf{center}\mathbf{r}_0^{\mathcal{H}} + \mathbf{r}_0^{\mathcal{I}}, \mathsf{center}\mathbf{r}_1^{\mathcal{H}} + \mathbf{r}_1^{\mathcal{I}})$$

where center derives a short value from the sum and *hash* is a so-called rational hash function as introduced in [BLNS23b]. The issuer then sends $\mathbf{t}$ to the issuer, together with a NIZK that it is well-formed according to $\mathbf{c}, \mathbf{r}_0^{\mathcal{I}}, \mathbf{r}_1^{\mathcal{I}}$ and that all involved values are short.

The signature scheme is based on a hash-and sign signature. It precomputes preimages of a matrix $\mathbf{A}$ using a trapdoor Gaussian preimage sampler over an NTRU-like lattice (Antrag/hybrid sampling) to instantiate the issuer's trapdoor signing step. The short preimage $\mathbf{u}$ can then be returned to the holder, who verifies that $\mathbf{t} = \mathbf{Au}$.

We illustrate the issuance transcript flow and pre-sign proofs in Figure 1.

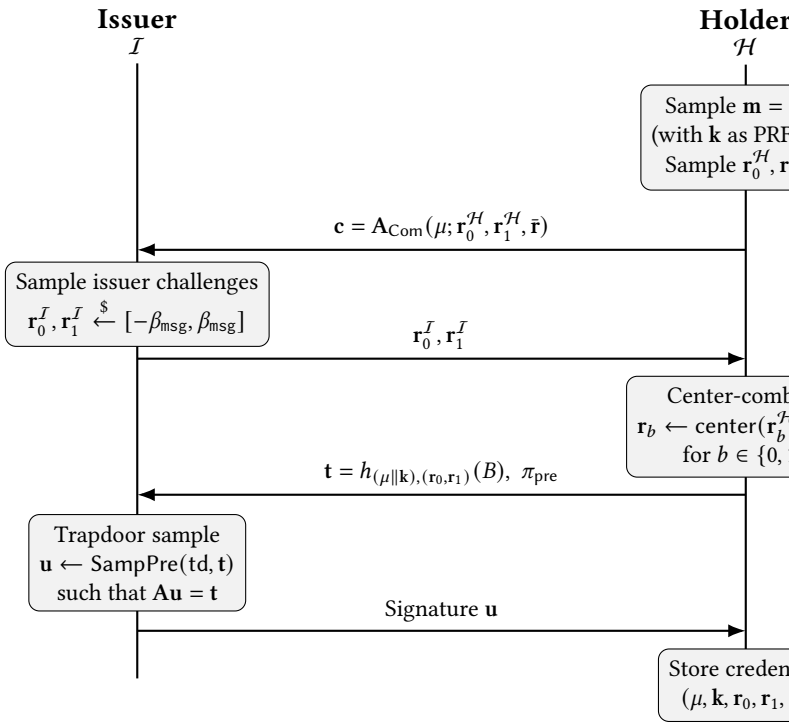

Figure 1: Issuance flow (blind signature). The pre-sign proof π_{pre} binds $\mathbf{t}$ to the commitment and the centered randomness, while hiding μ.

Blind Signatures and ARC Presentations. Let $\mathbf{r}_0 := \mathsf{center}(\mathbf{r}_0^{\mathcal{H}} + \mathbf{r}_0^{\mathcal{I}}), \mathbf{r}_1 := \mathsf{center}(\mathbf{r}_1^{\mathcal{H}} + \mathbf{r}_1^{\mathcal{I}})$. A presentation is a NIZK showing knowledge of $\mathbf{m}, \mathbf{r}_0, \mathbf{r}_1, \mathbf{t}$, such that $\mathbf{t} = \mathit{hash}(\mathbf{m}, \mathbf{r}_0, \mathbf{r}_1)$, as well as $\mathbf{u}$ such that $\mathbf{t} = \mathbf{Au}$ where $\mathbf{m}, \mathbf{r}_0, \mathbf{r}_1, \mathbf{u}$ are short. Towards constructing ARC, we additionally compute $\mathsf{tag} = \mathsf{F}(\mathbf{k}; \mathsf{nonce})$, append it to the NIZK and additionally prove in the NIZK that nonce is in the rate limiting interval and that tag is well-formed.

The verifier checks the NIZK proof and enforces the rate policy by maintaining server-side state keyed by (nonce, tag) (or an application-specific encoding thereof) to avoid double-spending.

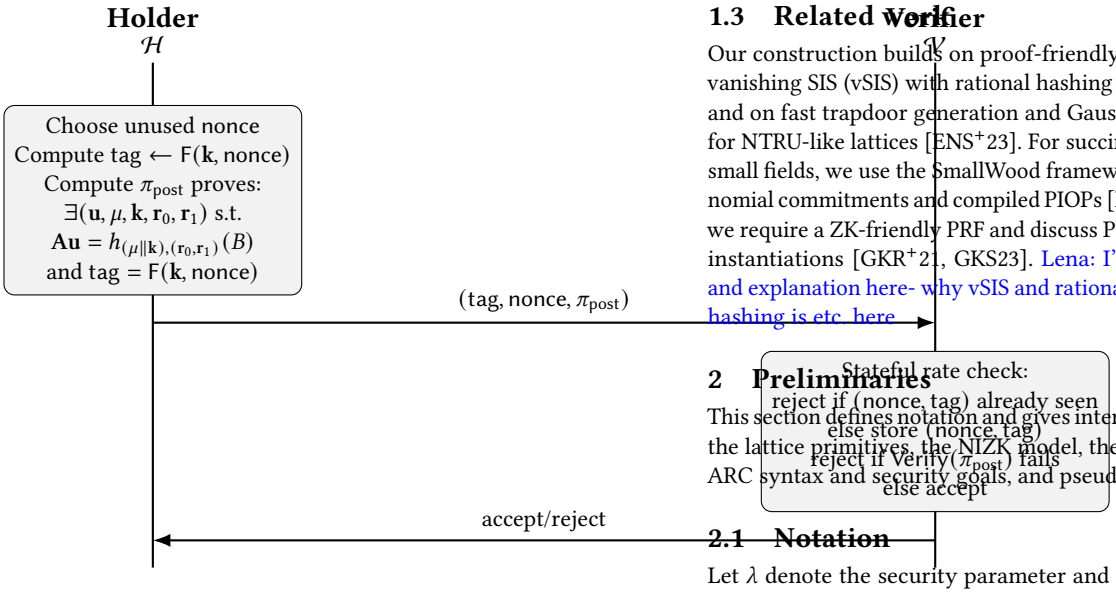

Figure 2: Showing (ARC presentation). Using an unused nonce ≤ presentation$_{\text{limit}}$**, the holder computes a deterministic** tag **from**(k, nonce) **(rate limiting) and pseudorandom across unused nonces (unlinkability).**

Figure 2 summarizes how tags enable stateful rate limiting without revealing the credential. Lena: I'm missing that the holder has some state in Figure 2

SmallWood Proofs. SmallWood [FR25] is a hash-based polynomial commitment scheme which produces efficient witnesses for polynomials of small degree $\leq 2^{16}$, which is particularly useful for lattice-based problems. The construction avoids the overhead of asymptotically-designed systems like STARKS while achieving better concrete efficiency than MPC-in-the-head variants for small witnesses. SmallWood exclusively relies on hash-based primitives, making it plausibly post-quantum secure. We use SmallWood as an NIZK for polynomial constraints over a small field $\mathbb{F}_q$, and we treat it as a black box that provides: (i) commitment to witness rows packed on an evaluation domain, (ii) succinct openings at verifier-chosen points, and (iii) soundness and zero knowledge after Fiat–Shamir [FR25].

To gain the best performance, the NIZK proofs for issuance and presentation require careful modeling of the proven statements in the NIZKs to argue how we optimized these. However, the SmallWood proof system is relatively new and has so far not been implemented.

We provide a uniform SmallWood programming model for the pre-sign and post-sign proofs, allowing us to easily describe witness row layouts and constraint families for the commitment, the function center, the rational hash *hash*, signature relation, and the evaluation checks for the Poseidon2 PRF. Proof-system internals (DECS/LVCS, PCS/PIOP, and soundness accounting) are deferred to Appendix C.

1.3 Related Work

Our construction builds on proof-friendly lattice signatures from vanishing SIS (vSIS) with rational hashing [DKLW25b, DKLW25a] and on fast trapdoor generation and Gaussian preimage sampling for NTRU-like lattices [ENS+23]. For succinct zero knowledge over small fields, we use the SmallWood framework for hash-based polynomial commitments and compiled PIOPs [FR25]. For tag derivation we require a ZK-friendly PRF and discuss Poseidon/Poseidon2-style instantiations [GKR+21, GKS23]. Lena: I'd add more motivation and explanation here- why vSIS and rational hashing, what rational hashing is etc. here

2 Preliminaries

This section defines notation and gives interface-level definitions for the lattice primitives, the NIZK model, the blind-signature syntax, ARC syntax and security goals, and pseudorandom functions.

2.1 Notation

Let λ denote the security parameter and $\text{negl}(\lambda)$ be any positive function negligible in λ. We write PPT for probabilistic algorithms whose runtime is polynomial in λ. For integers $a < b$ we write $[a, b] \coloneqq \{a, a+1, \ldots, b\}$ and we use $[b]$ as shorthand for $[1, b]$. Throughout this work, we denote vectors in lower-case bold letters such as $\mathbf{b}$ and matrices in upper-case bold letters such as $\mathbf{A}$.

We work over the cyclotomic ring $R = \mathbb{Z}[X]/(X^N + 1)$ for power-of-two N and its quotient $R_q = R/qR$, identified with polynomials of degree $< N$ with coefficients in $\mathbb{Z}_q$. For $a \in \mathbb{Z}$, we write $\langle a \rangle_q \in \{0, \ldots, q-1\}$ for reduction modulo q. When considering $\mathbb{Z}_q$-elements from an interval $[-a, b]$, we identify these with the set $\{q-a, q-a+1, \ldots, 0, 1, \ldots, b\}$. For elements $x \in R_q$ we write $||x|| \leq B$ to mean that each coefficient of x comes from $[-B, B]$. For vectors $\mathbf{x} \in R_q^\ell$, $||\mathbf{x}|| \leq B$ means that the statement holds for all elements of $\mathbf{x}$.

Remark on conflicting notation. Carsten: remove this before submission We write B for the *public* vSIS/BBS rational-hash key (and its cleared-denominator components B_0, B_1, B_2, B_3 when needed). The symbol β_{msg} denotes the *coefficient bound* used in sampling, membership constraints, and centering.

2.2 Commitment schemes and Ajtai commitments

Definition 2.1 (Commitment scheme)**.** Let $\mathcal{M}$ be a finite message space. A (non-interactive) commitment scheme is a tuple of PPT algorithms

$$\text{Com} = (\text{Setup}, \text{Commit}, \text{Verify})$$

with the following syntax:

- $\text{pp}_{\text{Com}} \leftarrow \text{Setup}(1^\lambda)$ is a PPT algorithm which outputs public parameters pp_{Com}. It is implicit input to all other algorithms.
- $c \leftarrow \text{Commit}(m)$ is a PPT algorithm which takes a message $m \in \mathcal{M}$ as input and outputs a commitment c and decommitment d.
- $\text{Verify}(c, m, d) \in \{0, 1\}$ deterministically checks validity of the opening.

Correctness holds if for all $\mathsf{pp}_{\mathsf{Com}} \leftarrow \mathsf{Setup}$, $m \in \mathcal{M}$ and $(c, d) \leftarrow \mathsf{Commit}(m)$ it holds that

$$\mathsf{Verify}(m, c, d) = 1.$$

The scheme is *computationally binding* if for all PPT adversaries $\mathcal{A}$,

$$\Pr_r\left[\begin{matrix}(\mathsf{pp}_{\mathsf{Com}}, c, m, d, m', d') \leftarrow \mathcal{A}(r) \wedge m \neq m' \\ \mathsf{Verify}(c, m, d) = 1 \wedge \mathsf{Verify}(c, m', d') = 1\end{matrix}\right] \leq \mathrm{negl}(\lambda).$$

The scheme is *(computationally) hiding* if for all PPT distinguishers $\mathcal{D}$, all messages $m_0, m_1 \in \mathcal{M}$ and $\mathsf{pp}_{\mathsf{Com}} \leftarrow \mathsf{Setup}()$,

$$\left|\Pr[\mathcal{D}^{O_{\mathsf{Commit}}(m_0)}(\mathsf{pp}_{\mathsf{Com}}) = 1] - \Pr[\mathcal{D}^{O_{\mathsf{Commit}}(m_1)}(\mathsf{pp}_{\mathsf{Com}}) = 1]\right| \leq \mathrm{negl}(\lambda)$$

where $O_{\mathsf{Commit}}(m_b)$ runs $(c, d) \leftarrow \mathsf{Commit}(m_b)$ and outputs c and the probability is taken over the randomness of Commit, $\mathcal{D}$.

2.2.1 Ajtai's Commitment scheme. Ajtai's commitment scheme considers R_q, the message length ℓ_m, randomness dimension ℓ_r, and output length ℓ_{Com} to be fixed. We then define the 3 algorithms as follows:

- $\mathsf{Setup}(1^\lambda)$ sets $\ell_{\mathsf{in}} = \ell_r + \ell_m$, samples and outputs a uniformly random matrix $\mathbf{A}_{\mathsf{Com}} \xleftarrow{\$} R_q^{\ell_{\mathsf{Com}} \times \ell_{\mathsf{in}}}$, chooses a bound $\beta_{\mathtt{msg}}$ and sets $\mathsf{pp}_{\mathsf{Com}} := (\mathbf{A}_{\mathsf{Com}}, \beta_{\mathtt{msg}})$. We note that
$$\mathcal{M} := \{\mathbf{m} \in R_q^{\ell_m} \mid ||\mathbf{m}|| \leq \beta_{\mathtt{msg}}\}.$$
- $\mathsf{Commit}(\mathbf{m})$ on input $\mathbf{m} \in \mathcal{M}$ uniformly samples $\mathbf{r} \in R_q^{\ell_r}$, $||\mathbf{r}|| \leq \beta_{\mathtt{msg}}$ and outputs $\mathbf{c} = \mathbf{A}_{\mathsf{Com}} \cdot [\mathbf{m} \| \mathbf{r}]^\top$ as well as $\mathbf{r}$.
- $\mathsf{Verify}(\mathbf{m}, \mathbf{c}, \mathbf{r})$ outputs 1 if
$$\mathbf{c} \in R_q^{\ell_{\mathsf{Com}}},\ \mathbf{r} \in R_q^{\ell_r},\ \mathbf{m} \in R_q^{\ell_m}, ||\mathbf{m}|| \leq \beta_{\mathtt{msg}},\ ||\mathbf{r}|| \leq \beta_{\mathtt{msg}}$$
and
$$\mathbf{c} = \mathbf{A}_{\mathsf{Com}} \cdot [\mathbf{m} \| \mathbf{r}]^\top$$
and 0 otherwise.

Definition 2.2 (SIS and ISIS over R_q)**.** Fix integers $n, m, \varphi \geq 1$ where φ is a power of 2 and let $q \geq 2$ be a modulus. Define R_q as above.

- **The homogeneous SIS problem.** Given $\mathbf{A} \in R_q^{n \times m}$ and a bound $\beta > 0$, find a non-zero $\mathbf{s} \in R^m$ such that $\mathbf{As} \equiv \mathbf{0} \pmod q$ and $\|\mathbf{s}\| \leq \beta$ (for a prescribed norm, e.g., ℓ_∞ or ℓ_2).
- **The inhomogeneous SIS (ISIS) problem.** Given $(\mathbf{A}, \mathbf{t}) \in R_q^{n \times m} \times R_q^n$ and $\beta > 0$, find $\mathbf{s} \in R^m$ with $\mathbf{As} \equiv \mathbf{t} \pmod q$ and $\|\mathbf{s}\| \leq \beta$.

We say that an attacker $\mathcal{A}$ solves the SIS problem if $\mathcal{A}$, on input $\mathbf{A}, \beta$ as well as all parameters of the SIS instance, outputs a valid solution $\mathbf{s}$. A similar definition can be made for the ISIS problem. On a formal level, we will assume that for the SIS/ISIS instances that we use, no PPT algorithm $\mathcal{A}$ solves them with non-negligible advantage. Concretely, we will later choose q, n, m, φ such that attacks require a runtime of at least 2^λ.

Carsten: Introduce ISIS problem here.

Remark 2.3 (Security intuition for Ajtai commitments). For uniform $\mathbf{A}_{\mathsf{Com}}$, binding reduces to finding a short non-zero kernel element of $\mathbf{A}_{\mathsf{Com}}$ with bound $2B$, i.e. to solve the SIS problem with the respective parameters. Hiding is obtained by choosing $\beta_{\mathtt{msg}}, \ell_r$ such that $\ell_r \geq \ell_{\mathsf{Com}} \cdot (\log_2(q)/(2\beta_{\mathtt{msg}}+1))$ so that $\mathbf{A}_{\mathsf{Com}}[\mathbf{0} \| \vec{r}]$ generates all elements in $R_q^{\ell_{\mathsf{Com}}}$ except with negligible probability, which statistically hides $\mathbf{m}$.

2.3 Lattices, trapdoors, vSIS, and rational hashing

We recall only the definitions necessary to define our construction; detailed implementation aspects e.g. of Gaussian sampler internals are deferred to Appendix B.

Trapdoor sampling interface. We use a trapdoor suite for sampling short preimages under a public linear map $\mathbf{A} \in R_q^{n \times m}$.

Definition 2.4 (Lattice trapdoor suite)**.** A lattice trapdoor suite for R_q, parameters m, n and an efficiently sampleable distribution $\mathcal{D}_{\mathsf{td}}$ over R_q^m consists of PPT algorithms (TrapGen, SampPre) such that:

- $(\mathbf{A}, \mathsf{td}) \leftarrow \mathsf{TrapGen}(1^\lambda)$ outputs a public matrix $\mathbf{A} \in R_q^{n \times m}$ and a trapdoor td.
- On input $\mathbf{A}, \mathsf{td}$ and a target $\mathbf{t} \in R_q^n$, SampPre outputs $\mathbf{u} \in R_q^m$ such that $\mathbf{t} = \mathbf{Au}$.

We require that $(\mathbf{Ar}, \mathbf{r} \leftarrow \mathcal{D}_{\mathsf{td}})$ is statistically indistinguishable from $(\mathbf{t} \leftarrow R_q^n, \mathsf{SampPre}(\mathbf{A}, \mathsf{td}, \mathbf{t}))$. Moreover, $\mathbf{A}$ as generated by TrapGen should be indistinguishable from uniformly random $\mathbf{A} \in R_q^{n \times m}$.

Lena: some of the domains are not clear to me,concretely the dimension of x_0, x_1 (ring elements?), and why b_1 is n-dimensional

Definition 2.5 (BBS-type rational hash and cleared-denominator form)**.** Let $(\mathbf{B}_0, b_1)$ be public parameters where $\mathbf{B}_0 \in R_q^{??}$ Carsten: add bounds and $\mathbf{b}_1 \in R_q^n$. Let the message be $\mathbf{m} \in R_q^{??}$ Carsten: add bounds and the hashing/signing randomness be $\chi = (\mathbf{x}_0, x_1) \in R_q^{???}$ Carsten: add bounds.

We define the BBS-type *rational hash*

$$h_{\mathbf{m},\chi}(\mathbf{B}_0, \mathbf{b}_1) \ := \ \mathbf{B}_0 \cdot (1 \| \mathbf{m} \| \mathbf{x}_0) \ \oslash \ (\mathbf{b}_1 - \mathbf{1}_n \cdot x_1),$$

where $\oslash$ is Hadamard (component-wise) division. The hash is well-defined only if each slot of $(\mathbf{b}_1 - \mathbf{1}_n \cdot x_1)$ is invertible.

Define $\mathbf{t} := h_{\mathbf{m},\chi}(\mathbf{B}_0, \mathbf{b}_1)$. Then the relation $\mathbf{As} \equiv \mathbf{t}$ over R_q is equivalent (assuming invertibility) to the *cleared-denominator identity*

$$(\mathbf{b}_1 - \mathbf{1}_n \cdot x_1) \odot (\mathbf{As}) \ - \ \mathbf{B}_0 \cdot (\mathbf{1} \| u \| \mathbf{x}_0) \ \equiv \ 0 \pmod q.$$

Distributing the Hadamard product yields a quadratic, proof-friendly form of total degree at most 2.

Definition 2.6 (Cleared-denominator rational hash)**.** Fix $B = (\mathbf{B}_0, \mathbf{B}_1, \mathbf{B}_2, \mathbf{B}_3)$ over R_q of compatible dimensions Carsten: which????. Given a message $\mathbf{m} \in R_q^{\ell_m}$ and randomness $(\mathbf{r}_0, \mathbf{r}_1)$, the hash output $\mathbf{t} \in R_q^n$ is defined as the unique value satisfying the *cleared-denominator identity*

$$(B_3 - r_1) \odot t \ - \ B_0 \ + \ B_1 \cdot m \ + \ B_2 \cdot r_0 \ = \ 0, \qquad (1)$$

whenever $(B_3 - r_1)$ is invertible component-wise in R_q. Equivalently,

$$t = \left(B_0 - B_1 \cdot m - B_2 \cdot r_0\right) \ \oslash \ (B_3 - r_1).$$

Carsten: this definition makes no sense to me. Are r_0, r_1 compatible with x_0, x_1? where do B_2, B_3 come from?? also, a hash has no randomness.

Remark 2.7 (How this matches BBS-style rational hashing). Definition 2.6 is a repackaging of the same "clear the denominator, keep degree ≤ 2" principle as in Definition 2.5, written in the $B =$

(B_0, B_1, B_2, B_3) parameterization used by ARC-SPRUCE. Carsten: how? this should be explained to the reader.

Definition 2.8 (vSIS-style hash-and-preimage signature template)**.** Fix a ring R_q and a Lattice trapdoor suite for parameters m, n and distribution $\mathcal{D}_{\text{td}}$. Let $\beta_{\text{sig}} \in \mathbb{N}$ be such that $\mathbf{r} \leftarrow \mathcal{D}_{\text{td}}$ implies $||\mathbf{r}|| \le \beta_{\text{sig}}$ except with negligible probability. Let $h_{\cdot,\cdot}$ be a BBS-type rational hash according to definition 2.5 with public key B. Let additionally $\mathcal{M}$ be a message space Carsten: define me.

Then we define the signature scheme vSIS = (KG, Verify, Verify) using the following algorithms:

- $\mathsf{KG}(1^\lambda)$ computes $(\mathbf{A}, \text{td}) \leftarrow \mathsf{TrapGen}(m, n, \mathcal{D}_{\text{td}})$. We let $\mathsf{vk} = \mathbf{A}$ and $\mathsf{sk} = \text{td}$.
- $\mathsf{Verify}(\mathbf{m}, \text{td})$ for $\mathbf{m} \in \mathcal{M}$ samples χ Carsten: specify distribution, computes $\mathbf{t} \leftarrow h_{\mathbf{m},\chi}$, $\mathbf{s} = \mathsf{SampPre}(\text{td}, \mathbf{t})$ and outputs $\sigma = (\mathbf{s}, \chi)$.
- $\mathsf{Verify}(\mathsf{vk}, \mathbf{m}, \sigma)$ where $\mathbf{m} \in \mathcal{M}$ and $\sigma = (\mathbf{s}, \chi)$ checks that

$$||\mathbf{s}|| \le \beta_{\text{sig}}, \ ||\chi|| \le \beta_{\text{sig}}$$

and

$$\mathbf{As} = h_{\mathbf{m},\chi}(\mathbf{B}).$$

Carsten: likely also need to check some condition on χ wrt B.

Carsten: can we make a statement about unforgeability based on previous work

2.4 Non-interactive zero-knowledge arguments of knowledge (NIZKs)

Lena: is a citation missing here?

Definition 2.9 (NIZKs in the random oracle model)**.** Let $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ be a family of relations. A (transparent) non-interactive zero-knowledge *argument of knowledge* for $\mathcal{R}$ in the random oracle model (with oracle $\mathsf{RO}(\cdot)$) is a pair of algorithms $(\mathsf{Prove}^{\mathsf{RO}}, \mathsf{Verify}^{\mathsf{RO}})$:

- $\mathsf{Prove}^{\mathsf{RO}}(w, x) \to \pi$ is probabilistic and on input (x, w) with $(x, w) \in R$ outputs a proof π.
- $\mathsf{Verify}^{\mathsf{RO}}(\pi, x) \to \{0, 1\}$ is a deterministic polynomial-time algorithm.

Completeness. For all $(x, w) \in \mathcal{R}$,

$$\Pr\left[\mathsf{Verify}^{\mathsf{RO}}(\pi, x) = 1 \ : \ \pi \leftarrow \mathsf{Prove}^{\mathsf{RO}}(w, x)\right] = 1.$$

Straightline-extractable knowledge soundness. There exists a PPT algorithm Ext, also called *extractor*, such that for any PPT adversary $\mathcal{A}$,

$$\Pr\left[\mathsf{Verify}^{\mathsf{RO}}(\pi, x) = 1 \ \wedge \ (x, w) \notin R\right] \ \le \ \varepsilon,$$

where $(\pi, Q) \leftarrow \mathcal{A}^{\mathsf{RO}}(x)$, Q is the set of query–response pairs made to RO by $\mathcal{A}$, and $w \leftarrow \mathsf{Ext}(Q, \pi)$.

Zero knowledge. There exists a PPT algorithm Sim, called the *simulator*, such that for any $(x, w) \in \mathcal{R}$ and PPT algorithm $\mathcal{A}$:

$$\left|\Pr[\mathcal{A}^{\mathsf{RO}}(\pi_{\text{real}}) = 1] - \Pr[\mathcal{A}^{\mathsf{RO}}(\pi_{\text{sim}}) = 1]\right| \le \xi,$$

where $\pi_{\text{real}} \leftarrow \mathsf{Prove}^{\mathsf{RO}}(w, x)$, $\pi_{\text{sim}} \leftarrow \mathsf{Sim}^{\mathsf{RO}}(x)$ and Sim can program the output of RO on any input.

We instantiate these proofs using the SmallWood PACS/PIOP framework compiled with Fiat–Shamir in the random oracle model [FR25]. SmallWood is a commit-and-prove proof system that only relies on symmetric primitives. While its proof size is linear in $|w|$ it is sublinear in the circuit size x.

2.5 Blind signatures

A blind signature algorithm allows a holder $\mathcal{H}$ to obtain signatures valid under the verification key of an issuer $\mathcal{I}$, without the latter learning the signed message μ. Our definitions follow [Fis06]. When any algorithm $\mathcal{B}$ uses the interactive protocol Issue with $\mathcal{I}$, we will write $\mathcal{B}^{\mathsf{Issue}:\langle \mathcal{I}(\cdot):\cdot\rangle}$ and adjust this the notation for other interactions accordingly.

Definition 2.10 (Blind Signature)**.** A Blind Signature scheme is a tuple of algorithms

$$\mathsf{BSig} = (\mathsf{Setup}, \mathsf{IssuerKeyGen}, \mathsf{Issue}, \mathsf{Verify})$$

such that:

- $\mathsf{Setup}(1^\lambda)$ is a PPT algorithm that outputs public parameters $\mathsf{pp}_{\mathsf{BSig}}$. All other algorithms obtain $1^\lambda, \mathsf{pp}_{\mathsf{BSig}}$ as implicit inputs.
- $\mathsf{IssuerKeyGen}()$ is a PPT algorithm that outputs issuer keys $(\mathsf{vk}, \mathsf{sk})$.
- Issue is an interactive protocol between $\mathcal{I}$ and $\mathcal{H}$, where $\mathcal{I}$ has input sk and $\mathcal{H}$ has input μ, vk. At the end of the protocol, $\mathcal{H}$ obtains sig.
- $\mathsf{Verify}(\mathsf{vk}, \mu, \mathsf{sig})$ is a deterministic polynomial-time algorithm that outputs a bit.

Definition 2.11 (Correctness)**.** A blind signature scheme is correct if there exists negl such that for all messages μ,

$$\Pr\left[\begin{array}{l}\mathsf{pp}_{\mathsf{BSig}} \leftarrow \mathsf{Setup}(1^\lambda);\\ (\mathsf{vk}, \mathsf{sk}) \leftarrow \mathsf{IssuerKeyGen}(\mathsf{pp}_{\mathsf{BSig}});\\ \mathsf{sig} \leftarrow \mathcal{H}^{\mathsf{Issue}:\langle \mathcal{I}(\mathsf{sk}):\cdot\rangle}(\mu, \mathsf{vk});\\ \mathsf{Verify}(\mathsf{vk}, \mu, \mathsf{sig}) = 1\end{array}\right] \ge 1 - \mathrm{negl}(\lambda).$$

Definition 2.12 (Blindness)**.** A blind signature scheme is blind if there exists negl such that for any PPT three-stage stateful adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ and any two messages μ_0, μ_1,

$$\left|\Pr\left[\begin{array}{l}\mathsf{pp}_{\mathsf{BSig}} \leftarrow \mathsf{Setup}(1^\lambda);\\ \mathsf{vk} \leftarrow \mathcal{A}_1(\mathsf{pp}_{\mathsf{BSig}});\\ b \xleftarrow{\$} \{0,1\};\\ \mathcal{A}_2^{\mathsf{Issue}:\langle \cdot:\mathcal{H}(\mu_b,\mathsf{vk})\rangle,\ \mathsf{Issue}:\langle \cdot:\mathcal{H}(\mu_{1-b},\mathsf{vk})\rangle};\\ \mathcal{H}(\mu_b, \mathsf{vk}) \text{ outputs } \mathsf{sig}_b;\\ \mathcal{H}(\mu_{1-b}, \mathsf{vk}) \text{ outputs } \mathsf{sig}_{1-b}\\ \text{if } \perp \in \{\mathsf{sig}_0, \mathsf{sig}_1\} \text{ then } (\mathsf{sig}_0, \mathsf{sig}_1) \leftarrow (\perp, \perp):\\ b = \mathcal{A}_3(\mathsf{sig}_0, \mathsf{sig}_1)\end{array}\right] - \frac{1}{2}\right| \le \mathrm{negl}(\lambda).$$

Definition 2.13 (One-more unforgeability)**.** A blind signature scheme is one-more unforgeable if there exists negl such that for any PPT adversary $\mathcal{A}^{\mathsf{Issue}:\langle \mathcal{I}(\mathsf{sk}):\cdot\rangle}(\mathsf{vk})$ that makes at most $Q = Q(\lambda)$ oracle queries to Issue,

$$\Pr\left[\begin{array}{l}\mathsf{pp}_{\mathsf{BSig}} \leftarrow \mathsf{Setup}(1^\lambda);\\ (\mathsf{vk}, \mathsf{sk}) \leftarrow \mathsf{IssuerKeyGen}(\mathsf{pp}_{\mathsf{BSig}});\\ \{(\mu_i, \mathsf{sig}_i)\}_{i\in[Q+1]} \leftarrow \mathcal{A}^{\mathsf{Issue}\langle \mathcal{I}(\mathsf{sk}):\cdot\rangle}(\mathsf{vk}):\\ \forall\, 1 \le i < j \le Q+1 : \ \mu_i \ne \mu_j \ \wedge \ \forall\, i \in [Q+1] : \ \mathsf{Verify}(\mathsf{vk}, \mu_i, \mathsf{sig}_i) = 1\end{array}\right] \le \mathrm{r}$$

2.6 Anonymous Rate-Limiting Credentials (ARCs)

Anonymous Rate-Limited credentials offer a similar syntax to Blind signatures. However, they additionally provide an algorithm Show that may create multiple presentations of a previous output of Issue. We also explicitly introduce the additional verifier party $\mathcal{V}$ that will obtain the presentations.

Definition 2.14 (ARC scheme syntax)**.** Let $\mathcal{N}$ be a finite set of nonces. An anonymous rate-limited credential (ARC) scheme is a tuple of algorithms

$$\mathsf{ARC} = (\mathsf{Setup}, \mathsf{IssuerKeyGen}, \mathsf{Issue}, \mathsf{Show}, \mathsf{Verify})$$

such that: Carsten: should there also be a message attached to the credential?

- $\mathsf{Setup}(1^\lambda)$ is a PPT algorithm that outputs public parameters $\mathsf{pp}_{\mathsf{ARC}}$. All other algorithms obtain $1^\lambda, \mathsf{pp}_{\mathsf{ARC}}$ as implicit inputs.
- $\mathsf{IssuerKeyGen}()$ is a PPT algorithm that outputs issuer keys $(\mathsf{vk}, \mathsf{sk})$.
- Issue is an interactive protocol between $\mathcal{I}$ and $\mathcal{H}$, where $\mathcal{I}$ has input sk and $\mathcal{H}$ has input vk. At the end of the protocol, $\mathcal{H}$ obtains cred.
- Show on input a credential cred and a nonce nonce $\in \mathcal{N}$ outputs a presentation pres. Lena: clarify where nonce comes from Carsten: hopefully done.
- $\mathsf{Verify}(\mathsf{vk}, \mathsf{pres}, \mathsf{nonce})$ is a deterministic polynomial-time algorithm that outputs a bit. Lena: where does pres; st come from? Carsten: skip state here and instead make it part of the security game.

Adjusting the correctness definition 2.11, blindness definition 2.12 and unforgeability definition 2.13 to ARC follows trivially. However, we additionally require that if the same credential cred is presented multiple times, but for different nonces from $\mathcal{N}$, then the presentations should be unlinkable. To formalize this, we let a challenger C create two credentials in interactions where the adversary controls the issuer. Then the challenger either creates two presentations for the same credential, or for different credentials.

Definition 2.15 (Presentation unlinkability)**.** Let ARC be an ARC scheme. Consider the following experiment $\mathsf{Exp}^{\mathsf{pres\text{-}unlink}}_{\mathsf{ARC},\mathcal{A}}(\lambda)$ between a challenger C and a two-stage stateful adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$:

(1) C runs $\mathsf{pp}_{\mathsf{ARC}} \leftarrow \mathsf{Setup}(1^\lambda)$, $(\mathsf{vk}, \mathsf{sk}) \leftarrow \mathsf{IssuerKeyGen}(\mathsf{pp}_{\mathsf{ARC}})$.
(2) Run $\mathcal{A}_1^{\mathsf{Issue}:\langle \cdot, C(\mathsf{vk})\rangle,\ \mathsf{Issue}:\langle C(\mathsf{vk}), \cdot\rangle}$, at the end of which C obtains $\mathsf{cred}_0, \mathsf{cred}_1$ and $\mathcal{A}_1$ outputs $\mathsf{nonce}_0, \mathsf{nonce}_1$.
(3) If cred_0 or cred_1 is $\perp$ or if $\mathsf{nonce}_0 = \mathsf{nonce}_1$ then output a random bit.
(4) C samples $b \xleftarrow{\$} \{0,1\}$ and computes

$$\mathsf{pres}_0 \leftarrow \mathsf{Show}(\mathsf{cred}_0, \mathsf{nonce}_0), \qquad \mathsf{pres}_1 \leftarrow \mathsf{Show}(\mathsf{cred}_b, \mathsf{nonce}_1).$$

Lena: that should be cred_1 on the second presentation, right?Carsten: no I think it makes sense

(5) $\mathcal{A}_2(\mathsf{pres}_0, \mathsf{pres}_1)$ outputs a bit b'.

Define $\mathcal{A}^{\mathsf{pres\text{-}unlink}}_{\mathsf{ARC}}(\mathcal{A}) := \left|\Pr[b' = b] - \frac{1}{2}\right|$. We say ARC has *presentation unlinkability* if for all PPT $\mathcal{A}$ this advantage is negligible in λ.

Message structure (global rule). Carsten: no idea where to put this. Every credential message is split as

$$m \ = \ m_1 \| m_2.$$

Here m_1 encodes holder attributes (potentially partially revealed in showings) and m_2 is sampled uniformly and serves as the *secret* PRF key used for rate limiting. We never swap these roles.

2.7 Pseudorandom functions

Definition 2.16 (Pseudorandom function (PRF))**.** Let $\mathcal{K}$, $\mathcal{D}$ and $\mathcal{T}$ be sets such that $|\mathcal{K}|$ is exponential in λ. A function family $\mathsf{F} : \mathcal{K} \times \mathcal{D} \to \mathcal{T}$ is a PRF if for all PPT adversaries $\mathcal{A}$,

$$\mathcal{A}^{\mathsf{prf}}_{\mathsf{F}}(\mathcal{A}) := \left|\Pr[\mathcal{A}^{O_0} = 1] - \Pr[\mathcal{A}^{O_1} = 1]\right| \le \mathrm{negl}(\lambda),$$

where $O_0(\cdot) = \mathsf{F}(k, \cdot)$ for uniform $k \xleftarrow{\$} \mathcal{K}$ and $O_1(\cdot)$ is a uniform random function from $\mathcal{D}$ to $\mathcal{T}$.

In this work, we instantiate PRFs based on the Poseidon2 construction Carsten: todo: citation.

It is based on the Poseidon2 permutation, which is defined as follows:

Definition 2.17 (Poseidon2-style permutation structure)**.** Let q be a prime, $d \ge 3$ such that $\gcd(d, q-1) = 1$ and $t, R_F, R_P \in \mathbb{N}$ where R_F is even. We call R_F the *external* rounds and R_P the *internal* rounds. Furthermore, let $\{c_j^{(i)}\}_{i\in[R_F]}^{j\in[t]} \subset \mathbb{F}_q$, $\{\hat{c}^{(i)}\}_{i\in[R_P]} \subset \mathbb{F}_q$ be public round constants and $\mathbf{M}_E, \mathbf{M}_I \in \mathbb{F}_q^{t\times t}$ be invertible maps.

For $\mathbf{x} = (x_1, \ldots, x_t) \in \mathbb{F}_q^t$, define external rounds

$$E_i(\mathbf{x}) := \mathbf{M}_E \cdot \left((x_1 + c_1^{(i)})^d, \ldots, (x_t + c_t^{(i)})^d\right)^\top,$$

and internal rounds

$$I_i(\mathbf{x}) := \mathbf{M}_I \cdot \left((x_1 + \hat{c}^{(i)})^d,\ x_2, \ldots, x_t\right)^\top.$$

Then

$$P := E_{R_F} \circ \cdots \circ E_{R_F/2+1} \circ I_{R_P} \circ \cdots \circ I_1 \circ E_{R_F/2} \circ \cdots \circ E_1.$$

We additionally define $\mathrm{Tr}_i : \mathbb{F}_q^t \to \mathbb{F}_q^i$ outputs the first i elements of the input vector.

Definition 2.18 (Poseidon2 PRF)**.** Let $\ell_{\mathsf{tag}}, \ell_{\mathsf{key}}, \ell_{\mathsf{in}} \in \mathbb{N}$ and set $t = \ell_{\mathsf{key}} + \ell_{\mathsf{nonce}}$. For a key $\mathbf{k} \in \mathbb{F}_q^{\ell_{\mathsf{key}}}$ and public input $\mathbf{n} \in \mathbb{F}_q^{\ell_{\mathsf{in}}}$, we define the Poseidon2 PRF as

$$\mathsf{F}(\mathbf{k}, \mathbf{n}) := \mathrm{Tr}_{\ell_{\mathsf{tag}}}\left(P(\mathbf{k}\|\mathbf{n}) + (\mathbf{k}\|\mathbf{n})\right),$$

where $P : \mathbb{F}_q^t \to \mathbb{F}_q^t$ is the permutation from Definition 2.17.

Constraint view (what the post-sign NIZK must prove). Carsten: All of this should go to a later section. To prove $\mathsf{tag} = \mathsf{F}(m_2, \mathsf{nonce})$ inside SmallWood, we arithmetize: (i) S-box constraints of the form $z = y^d$ at selected checkpoints, (ii) linear constraints for adding round constants and applying M_E, M_I between checkpoints, and (iii) feed-forward plus truncation constraints binding the public tag to the final state. The constraint families are stated in Section 5.5, while checkpoint scheduling and extended notes are in Appendix E.

3 Blind Signature Construction

This section presents the vSIS-based blind signature that underlies ARC-SPRUCE. The issuance protocol denotes the blind signature protocol, and its proof is the *pre-sign NIZK*.

3.0.1 Randomizing the hash funciton target. In the protocol, $\mathcal{H}$ will initially generate an Ajtai commitment to the committed vector

$$\mathbf{w} = [\mu \| \mathbf{k} \| \mathbf{r}_0^{\mathcal{H}} \| \mathbf{r}_1^{\mathcal{H}} \| \bar{\mathbf{r}}]^\top$$

where μ is the message, $\mathbf{k}$ a key (to be used later) and $\mathbf{r}_0^{\mathcal{H}}, \mathbf{r}_1^{\mathcal{H}}$ whose use we explain below. Additionally, $\bar{\mathbf{r}}$ is the randomness of the Ajtai commitment. To be binding, we require that $||\mathbf{w}|| \leq \beta_{\mathtt{msg}}$.

The issuance process relies on a two-party randomness generation step: the holder samples and commits to randomness $(\mathbf{r}_0^{\mathcal{H}}, \mathbf{r}_1^{\mathcal{H}})$ and the issuer sends an independent challenge $(\mathbf{r}_0^{\mathcal{I}}, \mathbf{r}_1^{\mathcal{I}})$. The holder then combines them using a public centering map center that removes bias:

$$\mathbf{r}_0 \;=\; \mathsf{center}(\mathbf{r}_0^{\mathcal{H}} + \mathbf{r}_0^{\mathcal{I}}), \qquad \mathbf{r}_1 \;=\; \mathsf{center}(\mathbf{r}_0^{\mathcal{H}} + \mathbf{r}_1^{\mathcal{I}}),$$

applied coefficient-wise. Concretely, center maps $[-2 \cdot \beta_{\mathtt{msg}}, 2 \cdot \beta_{\mathtt{msg}}]$ to $[-\beta_{\mathtt{msg}}, \beta_{\mathtt{msg}}]$ by wrapping modulo $\Delta = 2 \cdot \beta_{\mathtt{msg}} + 1$(see Appendix A for the explicit definitions). Lena: I'd write something linke 2*bound to make it explicit, seems like a new variable

3.0.2 vSIS/BBS rational hash. We use the cleared-denominator rational hash from Definition 2.6. In particular, the target $t = h_{m,(r_0,r_1)}(B)$ is characterized by the cleared-denominator identity Equation (1). The denominator invertibility is enforced by rejection sampling in the holder randomness generation, and the pre-sign NIZK proves that the target t was derived from bounded inputs and satisfies Equation (1).

3.0.3 Issuance protocol. We now state the full issuance protocol below. For simplicity, we already encompass $\mathbf{k}$ into the construction.

$\mathsf{Setup}(1^\lambda)$**:**

(1) Carsten: I guess we should say that the ring dimensions and q are fixed in advance.
(2) Compute and output $\mathsf{pp}_{\mathsf{Com}} \leftarrow \mathsf{Setup}(1^\lambda)$.
(3) Carsten: generate hash key B here, how? also, where does $\beta_{\mathtt{msg}}$ come from? Ajtai commitment must know length of vector to generate $\mathsf{pp}_{\mathsf{Com}}$.

$\mathsf{IssuerKeyGen}()$**:** $\mathcal{I}$ computes $(\mathbf{A}, \mathsf{td}) \leftarrow \mathsf{TrapGen}(1^\lambda)$. Let $\mathsf{vk} = (\mathbf{A})$ and $\mathsf{sk} = (\mathsf{td})$.

Issue: $\mathcal{I}$ holds sk while $\mathcal{H}$ has μ, vk.

(1) $\mathcal{H}$ samples $\mathbf{k}, \mathbf{r}_0^{\mathcal{H}}, \mathbf{r}_0^{\mathcal{H}}, \bar{\mathbf{r}}$ coefficient-wise in $[-\beta_{\mathtt{msg}}, \beta_{\mathtt{msg}}]$.
(2) $\mathcal{H}$ computes $\mathbf{c} \leftarrow \mathbf{A}_{\mathsf{Com}} \cdot [\mu \| \mathbf{k} \| \mathbf{r}_0^{\mathcal{H}} \| \mathbf{r}_0^{\mathcal{H}} \| \bar{\mathbf{r}}]^\top$ and sends $\mathbf{c}$ to $\mathcal{I}$.
(3) $\mathcal{I}$ samples $\mathbf{r}_0^{\mathcal{I}}, \mathbf{r}_1^{\mathcal{I}}$ coefficient-wise in $[-\beta_{\mathtt{msg}}, \beta_{\mathtt{msg}}]$ and sends $(\mathbf{r}_0^{\mathcal{I}}, \mathbf{r}_1^{\mathcal{I}})$ to $\mathcal{H}$.
(4) $\mathcal{H}$ sets $\mathbf{r}_0 \leftarrow \mathsf{center}(\mathbf{r}_0^{\mathcal{H}} + \mathbf{r}_0^{\mathcal{I}})$ and $\mathbf{r}_1 \leftarrow \mathsf{center}(\mathbf{r}_0^{\mathcal{H}} + \mathbf{r}_1^{\mathcal{I}})$.
(5) $\mathcal{H}$ computes $t \leftarrow h_{\mu \| \mathbf{k}, (\mathbf{r}_0, \mathbf{r}_1)}(B)$ and produces a *pre-sign* NIZK π_{pre} proving knowledge of $(\mu, \mathbf{k}, \mathbf{r}_0^{\mathcal{H}}, \mathbf{r}_0^{\mathcal{H}}, \bar{\mathbf{r}})$ such that:
   (a) $\mathbf{c} = \mathbf{A}_{\mathsf{Com}} \cdot [\mu \| \mathbf{k} \| \mathbf{r}_0^{\mathcal{H}} \| \mathbf{r}_0^{\mathcal{H}} \| \bar{r}]^\top$;
   (b) $\mathbf{t} = h_{\mu \| \mathbf{k}, (\mathsf{center}(\mathbf{r}_0^{\mathcal{H}} + \mathbf{r}_0^{\mathcal{I}}), \mathsf{center}(\mathbf{r}_0^{\mathcal{H}} + \mathbf{r}_1^{\mathcal{I}}))}(B)$;
   (c) all witness components are coefficient-wise in $[-\beta_{\mathtt{msg}}, \beta_{\mathtt{msg}}]$.
(6) $\mathcal{H}$ sends $(\mathbf{t}, \pi_{\mathsf{pre}})$ to $\mathcal{I}$.
(7) $\mathcal{I}$ verifies π_{pre}; if it accepts, sample $\mathbf{u} \leftarrow \mathsf{SampPre}(\mathsf{td}, \mathbf{t})$ and send $\mathbf{u}$ to $\mathcal{H}$. Otherwise abort.
(8) $\mathcal{H}$ checks $\mathbf{A}\mathbf{u} = \mathbf{t}$ and that $||\mathbf{u}|| \leq \beta_{\mathtt{sig}}$. If not, then abort.
(9) $\mathcal{H}$ produces a *post-sign* NIZK π_{post} proving knowledge of $(\mu, \mathbf{k}, \mathbf{u}, \mathbf{r}_0, \mathbf{r}_1)$ such that
   (a) $\mathbf{A}\mathbf{u} = h_{\mu \| \mathbf{k}, (\mathbf{r}_0, \mathbf{r}_1)}(B)$;
   (b) $\mu, \mathbf{k}, \mathbf{r}_0, \mathbf{r}_1$ are coefficient-wise in $[-\beta_{\mathtt{msg}}, \beta_{\mathtt{msg}}]$ and $||\mathbf{u}|| \leq \beta_{\mathtt{sig}}$.
(10) Set $\mathsf{sig} := (\pi_{\mathsf{post}})$.

$\mathsf{Verify}(\mathsf{vk}, \mu, \mathsf{sig})$**:** Let x be the post-sign verification statement for $\mu, \mathbf{k}, \beta_{\mathtt{msg}}, \beta_{\mathtt{sig}}$. For $\mathsf{sig} = (\pi)$ run $\mathsf{Verify}(\pi, x)$ and accept if the output is 1, otherwise reject.

3.0.4 Security.

Theorem 3.1. *Let B be such that the SIS problem is hard in Carsten: Specify ring and SIS parameters, $B, \ldots$ be such that the ISIS-f problem is hard Carsten: not defined yet, what are the security properties? and $\beta_{\mathtt{sig}}$ be such that Carsten: define security for trapdoor sampler. Then* $(\mathsf{Setup}, \mathsf{IssuerKeyGen}, \mathsf{Issue}, \mathsf{Verify})$ *is a blind signature.*

Proof.

Correctness. This follows directly from the construction and the completeness of the underlying NIZKs.

One-more unforgeability. Assume that there exists a PPT attacker $\mathcal{A}$ which wins the security game in Definition 2.13 with non-negligible probability. Thus, it outputs $Q+1$ signatures (μ_i, sig_i). We use the straight-line extractable knowledge soundness of the NIZK to extract $\mathbf{r}_{0,i}, \mathbf{r}_{1,i}, \mathbf{u}_i$ such that $||\mathbf{r}_{b,i}|| \leq B$ and $||\mathbf{u}_i|| \leq \beta_{\mathtt{sig}}$ and

$$\mathbf{A}\mathbf{u}_i = h_{\mu,(\mathbf{r}_{0,1}, \mathbf{r}_{1,i})}(B)$$

, except with negligible probability. Next, we let $\mathbf{A}$ be chosen by an instance of the ISIS-f experiment instead of using TrapGen. This is indistinguishable under Carsten: specify me. However, as we now cannot generate $\mathbf{u}_i$ anymore we let the ISIS-f challenger generate $\mathbf{r}_{b,i}$ as well as $\mathbf{u}_i$ for us, and use the knowledge soundness of the NIZK again to extract the $\mathbf{r}_{b,i}^{\mathcal{H}}$ from π_{pre} and choose $\mathbf{r}_{b,i}^{\mathcal{I}}$ accordingly. By assumption, one of the $(\mu_i, \mathbf{r}_{0,1}, \mathbf{r}_{1,i}, \mathbf{u}_i)$ cannot be generated by the ISIS-f challenger, but it still is a valid solution to ISIS-f, thus winning the security game.

Blindness. The adversary's view during issuance consists of $(\mathbf{c}, \mathbf{t}, \pi_{\mathsf{pre}})$. We show blindness by a hybrid argument. First, we let the challenger generate sig_i as well as π_{pre} using the simulator of the underlying NIZK. By the zero-knowledge property, this step is indistinguishable. Next, we let $\mathbf{c}$ be a commitment to a random vector instead of the message. By the hiding property of the commitment scheme, this is indistinguishable. Note that this means that π_{pre} now cannot prove correct opening of $\mathbf{c}$ anymore, which is indistinguishable as we are simulating the NIZK. Finally, we should reduce to the same problem where h hides μ as BLNS does.Carsten: add this!

□

4 ARC Construction from Blind Signatures

This section turns the blind signature of Section 3 into an anonymous rate-limited credential (ARC) system. We use the following terminology throughout: *issuance* refers to the blind signature protocol, while *showing* refers to an ARC presentation. The issuance proof is the *pre-sign NIZK*; the showing proof is the *post-sign NIZK*.

4.1 From blind signatures to ARC

A credential issued to a holder conceptually consists of:

- the signed message $m = \mu \| \mathbf{k}$, where μ are attributes and $\mathbf{k}$ is a uniformly sampled secret PRF key;
- the signature u satisfying $Au = h_{m,(r_0,r_1)}(B)$ for some bounded randomness (r_0, r_1).

The holder may additionally store local auxiliary values (e.g., the randomness (r_0, r_1)) needed to prove the signature relation in zero knowledge. These values are never revealed in showings.

4.2 Rate-limiting via the secret key k

Rate limiting is implemented by deterministic tags derived from a PRF keyed by the credential's secret $\mathbf{k}$. For a public nonce nonce $\in \mathcal{D}$, the holder computes

$$\text{tag} \;=\; \mathsf{F}(\mathbf{k}, \text{nonce}).$$

Intuitively:

- If the holder never repeats a nonce, then tag is pseudorandom and yields unlinkable showings.
- If the holder repeats a nonce, the tag repeats deterministically, enabling the verifier to detect double-spending for that nonce.

The verifier enforces its rate policy by restricting acceptable nonces (e.g., encoding epoch, service, or a bounded counter) and maintaining state that records previously accepted (nonce, tag) pairs for a given policy context.

4.3 Concrete PRF instantiation (Poseidon2-like)

We instantiate the PRF from Definition 2.16 using the Poseidon2-like construction of Definitions 2.18 and 2.17. Algorithm 1 summarizes the computation.

Algorithm 1 $\mathsf{F}(k, n)$ (Poseidon2-like with feed-forward + truncation)

$x \leftarrow k \| n$; $x^{(0)} \leftarrow x$
for $i = 1$ to $R_F/2$ **do**
  $x \leftarrow E_i(x)$
for $i = 1$ to R_P **do**
  $x \leftarrow I_i(x)$
for $i = R_F/2 + 1$ to R_F **do**
  $x \leftarrow E_i(x)$
$y \leftarrow x + x^{(0)}$ ▷ feed-forward
return $\mathrm{Tr}(y)$ ▷ first ℓ_{tag} lanes

Proof-friendly arithmetization. In the post-sign NIZK we enforce $\text{tag} = \mathsf{F}(\mathbf{k}, \text{nonce})$ using a checkpointed arithmetization: we commit selected S-box outputs and let the verifier replay all linear wiring between checkpoints from openings and public constants. Section 5.5 states the constraint families; Appendix E records the checkpoint schedule and extended notes.

4.4 Showing protocol

Algorithm 2 Show: ARC presentation (canonical protocol)

Require: Holder has credential $(\mu, \mathbf{k}, r_0, r_1, u)$. Public parameters include A, B, β_{msg} and PRF description.
Require: Input nonce nonce $\in \mathcal{D}$ that has not been used before for this credential.
Ensure: Presentation $(\text{tag}, \text{nonce}, \pi_{\text{post}})$.
1: Compute tag $\leftarrow \mathsf{F}(\mathbf{k}, \text{nonce})$.
2: Produce a *post-sign* NIZK π_{post} proving knowledge of $(u, \mu, \mathbf{k}, r_0, r_1)$ such that:
  (1) $Au = h_{\mu\|\mathbf{k},(r_0,r_1)}(B)$;
  (2) $\text{tag} = \mathsf{F}(\mathbf{k}, \text{nonce})$;
  (3) $\mu, \mathbf{k}, r_0, r_1$ are coefficient-wise in $[-\beta_{\text{msg}}, \beta_{\text{msg}}]$ and u satisfies the public shortness bound.
3: Output $(\text{tag}, \text{nonce}, \pi_{\text{post}})$.

The verifier runs Verify by checking π_{post} and consulting a policy state (e.g., a database) that tracks used nonces/tags. The verifier rejects if the proof fails or if the nonce/tag violates the policy (e.g., already spent, out of range, or malformed).

4.5 PRF role and rationale

The PRF serves two simultaneous roles:

- **Public rate-limiting handle.** The tag is deterministic in $(\mathbf{k}, \text{nonce})$ so the verifier can enforce stateful policies without interacting with the issuer and without learning the credential message.
- **Unlinkability across nonces.** For fresh nonces, tags are pseudorandom even given previous tags, so a verifier cannot link two presentations to the same credential unless the policy context forces reuse.

Crucially, $\mathbf{k}$ is part of the signed message, so the holder cannot change the PRF key between showings without forging a new signature. The post-sign NIZK binds the public tag to the signed $\mathbf{k}$ while keeping $\mathbf{k}$ hidden.

4.6 ARC security sketch

Correctness. If issuance completes and the holder uses a fresh nonce, then the post-sign statement is satisfiable by construction, by NIZK-ARK correctness (Definition 2.9) the verifier accepts. Lena: incomplete statement?

Unforgeability and policy soundness. By knowledge soundness (Definition 2.9), any accepting presentation yields a witness $(u, \mu, \mathbf{k}, r_0, r_1)$ satisfying both the signature/hash relation and the PRF-tag relation. Under one-more unforgeability of the blind signature (Section 3), an adversary cannot produce accepting showings for credentials it did not obtain via issuance. Policy soundness is enforced by verifier state: since $\text{tag} = \mathsf{F}(\mathbf{k}, \text{nonce})$ is uniquely determined by $(\mathbf{k}, \text{nonce})$, two accepting showings with the same nonce necessarily repeat the same tag, so a verifier that records spent pairs can reject replays.

Unlinkability. Presentation unlinkability is formalized in Definition 2.15. For distinct fresh nonces, PRF security (Definition 2.16) makes tags pseudorandom, while zero knowledge of the post-sign

proof (Definition 2.9) hides the credential secret. This yields indistinguishability of presentation transcripts, except for the intended nonce-reuse caveat in Definition 2.15.

Attribute privacy and selective disclosure. The message part μ can encode attributes; the post-sign NIZK can be extended to prove predicates about μ and to selectively reveal components, without changing the issuance protocol. We focus on the core rate-limiting construction and defer predicate engineering to the SmallWood programming model in Section 5.

5 SmallWood Programming Model

This section describes how ARC-SPRUCE statements are compiled into SmallWood constraints. Proof-system internals are deferred to Appendix C.

5.1 What SmallWood provides (formal interface)

Fix an evaluation set $\Omega = \{\omega_1, \ldots, \omega_s\} \subset \mathbb{F}_q$ of size s and a disjoint set of ℓ blinding points $\Omega' \subset \mathbb{F}_q \setminus \Omega$. A witness is arranged as a matrix $w \in \mathbb{F}_q^{n \times s}$, where each row w_i is interpolated into a polynomial $P_i(X) \in \mathbb{F}_q[X]$ such that $P_i(\omega_k) = w_{i,k}$ for all $\omega_k \in \Omega$ and P_i takes random values on Ω' (HVZK tails). SmallWood commits to the evaluations of all P_i and proves satisfaction of constraint polynomials evaluated on Ω, using random spot-checks outside Ω to enforce correctness with high probability.

Constraints: parallel vs aggregated. SmallWood's PACS language distinguishes two families of constraints over the packing domain Ω: (i) *parallel constraints* enforced pointwise for every $\omega \in \Omega$, and (ii) *aggregated constraints* enforced through a single Ω-sum identity of the form $\sum_{\omega \in \Omega} f'(\omega) = 0$. Parallel constraints are ideal for slot-local algebra (Hadamard products, linear relations, and checkpointed S-box constraints)Lena: I'd add that they are for Poseidon, while aggregated constraints are needed whenever the encoding introduces a *global* coupling across packed coordinates (e.g., row-sum identities arising from proof-friendly rewritings under packing, or global norm/shortness tests implemented via carry/limb aggregation).

In our ARC statements, the issuance proof is dominated by parallel constraints (commitment, centering, and the cleared-denominator hash relation). The showing proof uses parallel constraints for the signature/hash/PRF relations; for the concrete ARC instantiation in this paper we set $F_{\text{agg}} = \emptyset$, but we retain the distinction because proof-friendly encodings in earlier versions of the paper do use aggregated constraints to express global coupling. The concrete split between F_{par} and F_{agg} for our instantiation is spelled out below.

5.2 Constraint compilation recipe

Given a statement that relates ring elements (messages, randomness, signatures), we compile it as follows:

(1) **Choose witness rows.** Allocate one row per ring element (and per auxiliary carry/selector row if needed).
(2) **Fix packing.** Use coefficient packing on Ω: each column corresponds to one packed coordinate (e.g., one coefficient slot), and some rows may be constrained to be column-constant.
(3) **Introduce Θ-selectors.** For constraints that should apply at a single coordinate (e.g., binding an extracted PRF-key lane), define public selector polynomials that are 1 at one $\omega_k \in \Omega$ and 0 elsewhere.
(4) **Write constraint families.** Express all relations (commitment linearity, centering, rational-hash cleared identity, signature equation, PRF algebra) as polynomials in the witness rows and public Θ-values.
(5) **Add bounds via membership polynomials.** Enforce coefficient-wise bounds using a vanishing polynomial on $[-\beta_{\text{msg}}, \beta_{\text{msg}}]$ (Appendix A).

5.3 Issuance (pre-sign) constraints

We rewrite the pre-sign statement (Step 6 of **??**) as a SmallWood instance. The target T (the hash output t) is *public* in issuance, so the hash relation is linear in the witness.

Witness rows. We commit to the following fixed-order rows (each a ring element packed on Ω):

$$M_1,\ M_2,\ R_0^U,\ R_1^U,\ \bar{R},\ R_0,\ R_1,\ K_0,\ K_1.$$

Here M_1, M_2 are the message halves, $R_0^U, R_1^U, \bar{R}$ are holder randomnesses, R_0, R_1 are the centered combinations, and K_0, K_1 are carry rows for centering (coefficients in $\{-1, 0, 1\}$).

Public inputs. The verifier's public inputs are:

$$A_c,\ B,\ \text{com},\ r_0^I,\ r_1^I,\ T.$$

All public values are lifted to Θ-polynomials over Ω so that constraints are evaluated in the same domain as the witness rows.

Constraint families (mostly parallel). Let $\Delta = 2\beta_{\text{msg}} + 1$. The pre-sign proof enforces:

(1) **Commit constraints (linear):**

$$A_c \cdot [M_1 \| M_2 \| R_0^U \| R_1^U \| \bar{R}]^\top = \text{com}.$$

(2) **Center constraints (linear with carries):**

$$R_0 = R_0^U + r_0^I - \Delta \cdot K_0, \qquad R_1 = R_1^U + r_1^I - \Delta \cdot K_1,$$

enforced coefficient-wise on Ω, together with membership constraints $K_0, K_1 \in \{-1, 0, 1\}$.

(3) **Hash constraints (cleared denominator, linear in pre-sign):**

$$(B_3 - R_1) \odot T \ - \ B_0 \ + \ B_1 \cdot (M_1 + M_2) \ + \ B_2 \cdot R_0 \ = \ 0.$$

(4) **Packing constraints (message split).** We use half-split packing: M_1 occupies the lower half of coordinates and M_2 the upper half. We enforce $M_1(\omega_k) = 0$ for $k > s/2$ and $M_2(\omega_k) = 0$ for $k \le s/2$.
(5) **Bounds.** For each bounded row $X \in \{M_1, M_2, R_0^U, R_1^U, \bar{R}, R_0, R_1\}$, we enforce $P_{\beta_{\text{msg}}}(X(\omega_k)) = 0$ for all $\omega_k \in \Omega$, where $P_{\beta_{\text{msg}}}$ is the vanishing polynomial on $[-\beta_{\text{msg}}, \beta_{\text{msg}}]$. For K_0, K_1, we use the vanishing polynomial on $\{-1, 0, 1\}$.

5.4 Showing (post-sign) constraints

The showing proof (Step 2 of Algorithm 2) binds together the signature relation $A \cdot U = T$, the hash relation defining T, and the PRF relation defining tag. Unlike issuance, T is now a *witness row*, so the hash relation becomes bilinear.

Witness rows and ordering. We commit a single witness matrix whose rows are arranged as:

(1) **Base credential rows (as in issuance):** $M_1, M_2, R_0^U, R_1^U, \bar{R}, R_0, R_1, K_0, K_1$.
(2) **Hash target:** T (one row).
(3) **Signature preimage:** U (a small number of rows determined by the signature format; for an NTRU format this is two rows).
(4) **PRF witness block:** key-lane rows $\mathsf{Key}[0..(\ell_{\mathsf{key}}-1)]$ and checkpointed S-box output rows $\mathsf{Z}[0..(\mathsf{ZCount}-1)]$, as described in Section 5.5.

All PRF block rows are column-constant over Ω (up to the SmallWood tail masks) and are bound to the signed message M_2 via selector constraints.

Public inputs. The verifier receives A (signature matrix), B (hash key), the public tag and nonce, and the public bound β_{msg}.

Constraint families (parallel + aggregated). The post-sign proof enforces:

(1) **Signature constraint (linear):** $A \cdot U = T$.
(2) **Hash constraint (cleared denominator, bilinear):**

$$(B_3 - R_1) \odot T \;-\; B_0 \;+\; B_1 \cdot (M_1{+}M_2) \;+\; B_2 \cdot R_0 \;=\; 0.$$

(3) **PRF constraints.** We enforce:
- *Key binding:* each $\mathsf{Key}[i]$ equals a fixed public encoding of the signed secret $\mathbf{k}$ contained in M_2. This is enforced at a single Ω-coordinate using a Θ-selector polynomial.
- *Checkpointed S-box constraints:* for each committed checkpoint row $\mathsf{Z}[\alpha]$, enforce $\mathsf{Z}[\alpha] - \mathsf{In}[\alpha]^d = 0$, where $\mathsf{In}[\alpha]$ is the recomputed S-box input lane at that checkpoint (linear wiring is recomputed from opened evaluations and public constants).
- *Tag binding:* enforce the feed-forward plus truncation relation on the first ℓ_{tag} lanes, binding the public tag to the final PRF state.

(4) **Packing constraints (message split).** Same half-split constraints as in issuance.
(5) **Bounds.** Enforce coefficient-wise bounds for all bounded rows (message, randomness, target T, and signature rows U) using vanishing polynomials.

5.5 Poseidon PRF constraints with checkpoints

This subsection spells out the PRF part of the post-sign statement for the Poseidon2-like PRF of Section 4.3. All checks are expressed as *parallel* PACS constraints, i.e., they are enforced pointwise on the packing domain Ω (Section 5).

Witness rows for the PRF.. We allocate: (i) ℓ_{key} *key-lane* rows $\mathsf{Key}[0], \ldots, \mathsf{Key}[\ell_{\mathsf{key}}-1]$, and (ii) ZCount *checkpoint* rows $\mathsf{Z}[0], \ldots, \mathsf{Z}[\mathsf{ZCount}-1]$ holding committed S-box outputs. Each PRF row is column-constant on Ω (up to the HVZK tails on Ω'), so opening it at a single verifier point suffices to replay the PRF computation at that point.

Checkpoint schedule (grouping parameter g). Internal rounds are always checkpointed (one lane per round), while among external rounds we checkpoint every g-th round (all t lanes). Between two checkpoints, the verifier composes at most g S-box layers, so each checkpoint constraint has effective degree d^g (where d is the S-box exponent). A concrete schedule and the resulting row counts for our default instantiation are recorded in Appendix E.

Key binding (selector constraints). Let $\Theta_i(X)$ be a public selector polynomial that is 1 at the Ω-coordinate encoding the i-th key lane of the signed secret $\mathbf{k}$ inside the packed row M_2, and 0 elsewhere. We enforce:

$$\Theta_i(X) \cdot \big(\mathsf{Key}[i](X) - M_2(X)\big) = 0 \qquad \text{for all } i \in [0, \ell_{\mathsf{key}} - 1].$$

Since each $\mathsf{Key}[i]$ is column-constant on Ω, checking equality at a single selected coordinate binds it to the signed $\mathbf{k}$ value while keeping $\mathbf{k}$ hidden.

S-box checkpoint constraints. For each committed checkpoint α, the verifier recomputes the corresponding S-box input lane $\mathsf{In}[\alpha](X)$ (just before exponentiation) by replaying the linear wiring between checkpoints: adding public round constants and applying the public MDS matrices to previously committed checkpoint outputs and the key/nonce lanes. We enforce the parallel constraint

$$\mathsf{Z}[\alpha](X) - \big(\mathsf{In}[\alpha](X)\big)^d = 0 \qquad \text{for all } \alpha \in [0, \mathsf{ZCount} - 1].$$

Tag binding (feed-forward + truncation). Let $\mathbf{x}^{(0)} = \mathbf{k} \| \mathbf{n}$ be the initial state (with $\mathbf{k}$ given by the key-lane witness rows and $\mathbf{n}$ the public nonce encoding), and let $\mathbf{x}^{(\text{final})}$ be the recomputed final state after applying all rounds consistent with the checkpoint constraints. Define $\mathbf{y} = \mathbf{x}^{(\text{final})} + \mathbf{x}^{(0)}$. For each output lane $j \in [1, \ell_{\mathsf{tag}}]$, we enforce that the corresponding public tag component equals the truncated feed-forward state:

$$y_j(X) - \mathsf{tag}_j = 0,$$

where tag is the verifier's public input and y_j is the j-th lane of $\mathbf{y}$.

5.6 Where details are deferred

The SmallWood protocol stack (DECS/LVCS/PCS/PIOP), its soundness accounting, and the small-field variant are presented in Appendix C. The Gaussian trapdoor sampler for signing is detailed in Appendix B. The PRF algorithm is given in Section 4.3 and the PRF constraint families (including checkpointing) are stated in Section 5.5. Appendix E records extended notes (e.g., parameter-generation guidelines).

6 Parameters and Integration

This section explains how parameters interact across the ARC-SPRUCE stack: the lattice signature (trapdoor and preimages), the rational hash, the SmallWood proof system, and the PRF. Detailed tables and extended benchmarks are deferred to Appendix D.

6.1 Parameter dependencies

All components share the same base ring R_q and modulus q:

- The signature map $A \in R_q^{n \times m}$ and hash key B are defined over R_q.
- SmallWood constraints are evaluated over the base field $\mathbb{F}_q$ (coefficients modulo q) and therefore must use the same q.
- The PRF is instantiated over $\mathbb{F}_q$ so that its constraints are native to the same field.

The coefficient bound $\beta_{\mathtt{msg}}$ governs: (i) sampling ranges for $\mu, \mathbf{k}, r_0^U, r_1^U$ and issuer challenges, (ii) the centering modulus $\Delta = 2\beta_{\mathtt{msg}} + 1$, and (iii) membership polynomials used in the NIZKs.

6.2 Sampler parameters

The trapdoor sampler must output short signatures u with overwhelming probability and with a distribution close to a discrete Gaussian over the appropriate lattice coset. This requires choosing:

- ring degree N and modulus q,
- trapdoor quality (e.g., an NTRU trapdoor parameter α),
- per-slot Gaussian widths σ (or $\sigma_k[i]$ for a two-plane sampler),
- an acceptance predicate (norm bound) that yields a public verification bound $\beta_{\mathtt{sig}}$.

We use an Antrag-based NTRU trapdoor and a hybrid two-plane Gaussian sampler; details and tuning rules are given in Appendix B.

6.3 vSIS / rational hash parameters

The rational hash must satisfy:

- **Denominator invertibility.** The denominator $B_3 - r_1$ must be invertible component-wise; the holder enforces this by rejection sampling r_1 (and the NIZK proves boundedness and the cleared relation).
- **No wrap-around in bounds.** Membership checks for $[-\beta_{\mathtt{msg}}, \beta_{\mathtt{msg}}]$ are implemented via vanishing polynomials over $\mathbb{F}_q$, so we require $\beta_{\mathtt{msg}} \ll q$ to avoid modular wrap-around artifacts.
- **Compatibility with signature hardness.** The vSIS analyses [DKLW25b, DKLW25a] impose conditions on the rational hash family (e.g., strong linear independence and denominator bounds) which translate into constraints on the distribution of bounded randomness and on the public key structure.

6.4 SmallWood parameters

SmallWood introduces parameters controlling packing, masking, and soundness:

- **Packing size** $s = |\Omega|$ (often chosen as $s = n_{\text{cols}}$).
- **Tail length** ℓ for HVZK masking and degree enforcement.
- **Spot-check count** ℓ' and number of masks ρ for PIOP soundness.
- **Degree bounds** $d_{\text{decs}} = s + \ell - 1$ for witness rows and d_Q for constraint-derived masks.
- **Small-field knob** θ when using an extension-field variant.

The pre-sign proof is dominated by linear constraints (commitment, centering, and a linear cleared-denominator hash), while the post-sign proof introduces higher effective degree due to the PRF's S-box constraints and the bilinear hash relation (since T becomes a witness row). Appendix C and Appendix D detail how these degrees translate into d_Q and final soundness.

6.5 PRF parameters

The PRF must be: (i) secure at target level λ, (ii) efficiently representable in constraints, and (iii) compatible with $\mathbb{F}_q$. We use a Poseidon2-like permutation with feed-forward and truncation as a concrete instantiation. Its parameters include width $t = \ell_{\mathsf{key}} + \ell_{\mathsf{nonce}}$, round counts (R_F, R_P), MDS matrices, round constants, S-box exponent d, and truncation length ℓ_{tag}. We require $\ell_{\mathsf{tag}} \geq \lceil \lambda / \log_2(q) \rceil$ so the tag contains λ bits of entropy. The algorithm is given in Section 4.3, while the checkpointed constraint shape used in the post-sign proof is stated in Section 5.5. Appendix E records extended notes (optional).

6.6 Sizes and performance (example instantiation)

To ground the parameter interaction, we record one concrete demo instantiation (ring degree $N = 1024$, modulus $q = 1038337$, target $\lambda \approx 128$ bits) and report measured proof sizes for the two NIZKs. The pre-sign proof uses only the base witness rows, while the post-sign proof adds signature and PRF rows as described in Section 5.4.

Proof	Rows (base / extra)	Size	Soundness (bi
Pre-sign NIZK (issuance)	9 / 0	≈ 7.4 KB	≈ 129.6
Post-sign NIZK (showing)	12 / 86	≈ 20.0 KB	≈ 129.4

Table 1: Example proof sizes and row counts for one instantiation; see Appendix D for parameter tuples and extended data.

References

[AGJ+24] Sven Argo, Tim Güneysu, Corentin Jeudy, Georg Land, Adeline Roux-Langlois, and Olivier Sanders. Practical post-quantum signatures for privacy. pages 1523–1537, 2024.

[BLNS23a] Ward Beullens, Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Gregor Seiler. Lattice-based blind signatures: Short, efficient, and round-optimal. pages 16–29, 2023.

[BLNS23b] Jonathan Bootle, Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Alessandro Sorniotti. A framework for practical anonymous credentials from lattices. pages 384–417, 2023.

[DKLW25a] Adrien Dubois, Michael Klooß, Russell W. F. Lai, and Ivy K. Y. Woo. Lattice-based proof-friendly signatures from vanishing short integer solutions. pages 452–486, 2025.

[DKLW25b] Adrien Dubois, Michael Klooß, Russell W. F. Lai, and Ivy K. Y. Woo. Lattice-based proof-friendly signatures from vanishing short integer solutions. Cryptology ePrint Archive, Report 2025/356, 2025.

[ENS+23] Thomas Espitau, Thi Thu Quyen Nguyen, Chao Sun, Mehdi Tibouchi, and Alexandre Wallet. Antrag: Annular NTRU trapdoor generation. Cryptology ePrint Archive, Report 2023/1335, 2023.

[Fis06] Marc Fischlin. Round-optimal composable blind signatures in the common reference string model. pages 60–77, 2006.

[FR25] Thibauld Feneuil and Matthieu Rivain. SmallWood: Hash-based polynomial commitments and zero-knowledge arguments for relatively small instances. Cryptology ePrint Archive, Report 2025/1085, 2025.

[GKR+21] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. Poseidon: A new hash function for zero-knowledge proof systems. pages 519–535, 2021.

[GKS23] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A faster version of the poseidon hash function. pages 177–203, 2023.

[GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. pages 197–206, 2008.

[LSS24] Vadim Lyubashevsky, Gregor Seiler, and Patrick Steuer. The LaZer library: Lattice-based zero knowledge and succinct proofs for quantum-safe privacy. pages 3125–3137, 2024.

[PWFW23] Guru-Vamsi Policharla, Bas Westerbaan, Armando Faz-Hernández, and Christopher A Wood. Post-quantum privacy pass via post-quantum anonymous credentials. Cryptology ePrint Archive, Report 2023/414, 2023.

[SK25] Samuel Schlesinger and Jonathan Katz. Anonymous Credit Tokens. Internet-Draft draft-schlesinger-cfrg-act-00, Internet Engineering Task Force, August 2025. Work in Progress.

[YW25] Cathie Yun and Christopher A. Wood. Anonymous Rate-Limited Credentials. Internet-Draft draft-yun-privacypass-crypto-arc-00, Internet Engineering Task Force, October 2025. Work in Progress.

A Additional Definitions

This appendix collects definitions used by the main text.

A.1 Centering map and carries

Let $\mathsf{BoundB} \in \mathbb{Z}_{>0}$ and $\Delta = 2\mathsf{BoundB}+1$. Define the centering map $\mathrm{center} : [-2\mathsf{BoundB}, 2\mathsf{BoundB}] \to [-\mathsf{BoundB}, \mathsf{BoundB}]$ by

$$\mathrm{center}(x) = \begin{cases} x & \text{if } x \in [-\mathsf{BoundB}, \mathsf{BoundB}], \\ x+\Delta & \text{if } x \in [-2\mathsf{BoundB}, -\mathsf{BoundB}-1], \\ x-\Delta & \text{if } x \in [\mathsf{BoundB}+1, 2\mathsf{BoundB}]. \end{cases}$$

Applied coefficient-wise, this map allows unbiased combination of two bounded values when at least one is uniform.

Lemma A.1 (Centering preserves uniformity)**.** *Fix any $a \in [-\mathsf{BoundB}, \mathsf{BoundB}]$. If $b \xleftarrow{\$} [-\mathsf{BoundB}, \mathsf{BoundB}]$ then $\mathrm{center}(a+b)$ is uniform over $[-\mathsf{BoundB}, \mathsf{BoundB}]$.*

A.2 Vanishing polynomials for membership constraints

Membership constraints are expressed using vanishing polynomials over $\mathbb{F}_q$ with signed lifting.

Definition A.2 (Vanishing polynomial for $[-\mathsf{BoundB}, \mathsf{BoundB}]$)**.** Let $\mathsf{BoundB} \in \mathbb{Z}_{>0}$. Define

$$P_{\mathsf{BoundB}}(X) \;=\; \prod_{i=-\mathsf{BoundB}}^{\mathsf{BoundB}} \big(X - \langle i \rangle_q\big) \;\in\; \mathbb{F}_q[X].$$

For $x \in \mathbb{F}_q$, we have $x \in [-\mathsf{BoundB}, \mathsf{BoundB}]$ (via signed lifting) iff $P_{\mathsf{BoundB}}(x) = 0$ in $\mathbb{F}_q$.

Definition A.3 (Vanishing polynomial for $\{-1, 0, 1\}$)**.** Define $P_1(X) = (X+1)X(X-1) \in \mathbb{F}_q[X]$. Then $P_1(x) = 0$ iff $x \in \{-1, 0, 1\}$.

B Gaussian Preimage Sampling

This appendix details the Gaussian preimage sampler used by the issuer to sign a target $t \in R_q^n$ by producing a short preimage $u \in R^m$ such that $Au = t$ in R_q^n. Our instantiation follows the vSIS–BBS setting [DKLW25b] and relies on an NTRU trapdoor generated via Antrag together with a hybrid two-plane sampler [ENS+23].

B.1 The preimage sampling problem

Given a public linear map $A \in R_q^{n \times m}$ and a target $t \in R_q^n$, the signer samples

$$u \;\leftarrow\; \mathsf{SampPre}(\mathsf{td}_A, t) \qquad \text{such that} \qquad Au \equiv t \pmod q$$

and u is short (according to a public bound $\beta_{\mathtt{sig}}$). Equivalently, u lies in the coset lattice

$$\Lambda_\perp^t(A) \coloneqq \{\, x \in R^m : Ax \equiv t \pmod q \,\}.$$

We require: (i) correctness ($Au = t$ and the norm test fails only with negligible probability), (ii) (approx.) Gaussianity of u over the coset (statistical hiding of the trapdoor), and (iii) a tight tail bound that supports efficient ZK range/norm checks in the post-sign proof.

B.2 NTRU trapdoor form and quality

We work with the rank-2 NTRU module over the cyclotomic ring $R = \mathbb{Z}[X]/(X^N+1)$ and its quotient R_q. An NTRU public key has the form $h = gf^{-1} \bmod q$ for secret $f, g \in R$. The corresponding module is

$$\mathsf{L}_{\mathsf{NTRU}}(h) \;=\; \{(u, v) \in R^2 : uh - v \equiv 0 \pmod q\}.$$

A trapdoor basis of $\mathsf{L}_{\mathsf{NTRU}}(h)$ is

$$B_{f,g} \;=\; \begin{pmatrix} f & F \\ g & G \end{pmatrix}, \qquad \text{with} \qquad fG - gF \;=\; q,$$

whose columns are short and satisfy $g/f \equiv G/F \equiv h \pmod q$. Following [ENS+23], we measure trapdoor quality via per-slot energies under the canonical embeddings φ_i, requiring a uniform window around q. Concretely, for a quality parameter $\alpha > 1$ we target

$$\frac{q}{\alpha^2} \;\le\; |\varphi_i(f)|^2 + |\varphi_i(g)|^2 \;\le\; \alpha^2 q \qquad \text{for all slots } i.$$

Smaller α corresponds to tighter trapdoors and, for fixed N, q, to shorter signatures for the same security.

B.3 Sampler definition and tuning

Our sampler is a ring-based hybrid two-plane sampler in the evaluation domain. The sampler's main tuning parameters are:

- modulus q and ring degree N,
- trapdoor quality α,
- acceptance radius R_{acc} (controls rejections and ℓ_∞/ℓ_2 tails),
- per-slot widths $\sigma_k[i]$ (two planes $k \in \{1, 2\}$).

Per-slot calibration. Let $\widetilde{b}_k[i]$ denote the per-slot Gram–Schmidt lengths for plane k at slot i. A standard calibration equalizes acceptance probability across slots:

$$\sigma_k[i] \;=\; \sqrt{\frac{\alpha^2 q\, R_{\mathrm{acc}}^2}{\|\widetilde{b}_k[i]\|^2} \;-\; R_{\mathrm{acc}}^2} \qquad (k \in \{1, 2\}).$$

This prevents weak slots and yields consistent output variance.

Hiding threshold (smoothing). To achieve statistical hiding, we enforce a floor $\sigma_{\min}$ such that $\sigma_k[i] \ge \sigma_{\min}$ for all k, i, where $\sigma_{\min}$ exceeds the smoothing parameter of the relevant lattice cosets. In practice, we apply a global scale factor to raise all $\sigma_k[i]$ above the theoretical floor with slack.

B.4 Annulus sampling (Antrag key generation)

Antrag replaces trial-and-error trapdoor generation with direct annulus sampling in the Fourier domain [ENS+23]. For each conjugate pair (there are $N/2$ independent slots), the key generator samples $(|\varphi_i(f)|, |\varphi_i(g)|)$ uniformly in an annulus arc $A^+(r, R)$ with

$$\sqrt{q}/\alpha \;<\; r \;<\; R \;<\; \alpha\sqrt{q},$$

chooses random phases, inverse-embeds to the coefficient domain, and rounds to obtain $(f, g) \in R^2$. By selecting a "middle" annulus window (see [ENS+23]) one ensures that the rounded trapdoor satisfies the desired α-window with high probability and with modest repetition.

B.5 Hybrid two-plane sampler (details)

We sketch the two-plane hybrid sampler in the notation of [ENS+23]. Let td_A encode an NTRU trapdoor for A and let $t \in R_q^n$ be the target. Write $\langle\cdot\rangle_q$ for signed coefficient lift and define center_q as coefficient-wise centering into $(-\frac{q}{2}, \frac{q}{2}]$.

Algorithm 3 $\mathrm{SampPre}(\mathrm{td}_A, t)$: two-plane hybrid sampler (sketch)

1: Lift t to coefficients and set $c \leftarrow \mathrm{center}_q(t) \in (-\frac{q}{2}, \frac{q}{2}]^n$.
2: Using td_A, compute per-slot widths $\sigma_k[i]$ (two planes) calibrated to trapdoor quality.
3: **repeat**
4: Sample slot-wise Gaussian pairs on the two planes, reconstruct a coefficient-domain candidate u.
5: Accept iff the norm predicate holds for u (e.g., ℓ_2 and/or ℓ_∞ bound).
6: **until** accepted
7: **return** u.

Public relations. Downstream verification and proofs use only the public relation $Au = t$ and shortness of u (bounded by $\beta_{\mathtt{sig}}$), checked by the holder on receipt and enforced by the issuer's rejection sampling. The post-sign NIZK then proves knowledge of such a short u without revealing it.

C SmallWood Proof-System Details

This appendix recalls the SmallWood proof stack and provides algorithmic details used by our instantiations. The main text treats SmallWood as a black box; here we summarize the DECS/LVCS/PCS/PIOP layers and the Fiat–Shamir compilation, following [FR25].

C.1 Domains and packing

Fix a base field $\mathbb{F}_q$, an evaluation set $\Omega \subset \mathbb{F}_q$ of size s, and a disjoint tail set $\Omega' \subset \mathbb{F}_q \setminus \Omega$ of size ℓ. A witness row is interpolated into a polynomial of degree $\le s + \ell - 1$ whose values on Ω are the packed coordinates and whose values on Ω' are random masks (HVZK).

C.2 DECS (Degree-Enforcing Commitment Scheme)

DECS commits to evaluations of witness polynomials and enforces that they come from low-degree polynomials rather than arbitrary vectors. Commitments are implemented via a Merkle tree over an NTT-sized domain, and degree soundness is obtained by checking masked low-degree relations at verifier-chosen indices.

Algorithm 4 $\mathrm{DECS.DeriveGamma}(\mathrm{root}, \eta, r, q) \to \Gamma$ (verifier and prover)

1: Initialize counter $ctr \leftarrow 0$; let $L \leftarrow \left\lfloor \frac{2^{64}}{q} \right\rfloor \cdot q$.
2: **for** $k = 0$ to $\eta - 1$ **do**
3: **for** $j = 0$ to $r - 1$ **do**
4: **repeat**
5: $x \leftarrow \mathrm{SHA256}(\mathrm{root} \,\|\, \mathrm{LE64}(ctr))$ interpreted as a 64-bit integer
6: $ctr \leftarrow ctr + 1$
7: **until** $x < L$
8: $\Gamma_{k,j} \leftarrow x \bmod q$
9: **return** Γ

Algorithm 5 $\mathrm{DECS.VerifyEval}(\mathrm{root}, \Gamma, R, \mathrm{open})$ (verifier)

Require: Domain size N, modulus q, mask count η, committed row count r.
1: **for** each opened index e **do**
2: Verify Merkle authentication for e against root; reject on failure.
3: **for** each mask row $k < \eta$ **do**
4: Check $R_k^\star[e] \stackrel{?}{=} M_k(e) + \sum_{j=0}^{r-1} \Gamma_{k,j} P_j(e) \pmod q$.
5: **return** accept

C.3 LVCS (Linear Vector Commitments)

LVCS commits to packed witness rows and supports succinct openings of $\mathbb{F}_q$-linear forms of those rows. SmallWood uses LVCS both as a routing/batching layer and as the core of its hash-based PCS: the verifier requests openings of linear forms that correspond to evaluating constraint residuals at verifier-chosen points, and DECS enforces that opened values come from low-degree polynomials.

Algorithm 6 $\mathrm{LVCS.CommitInitWithParams}(\mathrm{ringQ}, \{r_j\}_{j<r}, \ell, \mathrm{params}) \to (\mathrm{root}, \mathrm{ProverKey})$ (prover)

Require: Ring R_q of size N, packed rows $r_j \in \mathbb{F}_q^{n_{\mathrm{cols}}}$, tail length $\ell > 0$.
1: Sample $\mathrm{mask}_j \xleftarrow{\$} \mathbb{F}_q^\ell$ for each row j (HVZK tails).
2: Interpolate each row to $P_j(X) \leftarrow \mathrm{interpolateRow}(\mathrm{ringQ}, r_j, \mathrm{mask}_j, n_{\mathrm{cols}}, \ell)$.
3: Commit to $\{P_j\}$ via DECS and obtain Merkle root root.
4: Derive $\Gamma \leftarrow \mathrm{DECS.DeriveGamma}(\mathrm{root}, \eta, r, q)$ and cache values needed for openings.
5: **return** $(\mathrm{root}, \mathrm{ProverKey})$.

Algorithm 7 $\mathrm{LVCS.CommitStep1}(\mathrm{root}) \to \Gamma$ (verifier)

1: Store root and derive $\Gamma \leftarrow \mathrm{DECS.DeriveGamma}(\mathrm{root}, \eta, r, q)$.
2: **return** Γ.

Algorithm 8 LVCS.CommitStep2($\{R_k\}_{k<\eta}$) $\to$ accept/reject (verifier)

1: **for** each mask row R_k **do**
2: Reject if $\deg(R_k) > d$ (degree guard before openings).
3: **return** accept.

Algorithm 9 LVCS.EvalInitMany(ringQ, ProverKey, $\{\text{Coeffs}_k\}_{k<m}$) $\to \{\bar{v}_k\}_{k<m}$ (prover)

Require: Linear forms $\text{Coeffs}_k \in \mathbb{F}_q^r$ for $k = 0, \ldots, m-1$.
1: Let $\ell \leftarrow |\text{mask}_j|$.
2: **for** $k = 0$ to $m-1$ **do**
3: Initialize $\bar{v}_k \leftarrow 0 \in \mathbb{F}_q^\ell$.
4: Set $\bar{v}_k \leftarrow \sum_{j=0}^{r-1} \text{Coeffs}_k[j] \cdot \text{mask}_j \in \mathbb{F}_q^\ell$.
5: **return** $\{\bar{v}_k\}_{k<m}$.

Algorithm 10 LVCS.ChooseE(ℓ, n_{cols}) $\to E$ (verifier)

Require: Ring size N; packed domain size $n_{\text{cols}} = |\Omega|$; masked slab indices $[n_{\text{cols}}, n_{\text{cols}} + \ell)$.
1: Sample $E \subset \{n_{\text{cols}}+\ell, \ldots, N-1\}$ uniformly without replacement with $|E| = \ell$.
2: **return** E.

Algorithm 11 LVCS.EvalFinish(ProverKey, E) $\to$ Opening (prover)

1: open $\leftarrow$ ProverKey.DecsProver.EvalOpen(E).
2: **return** Opening{DECSOpen = open}.

Algorithm 12 LVCS.EvalStep2($\{\bar{v}_k\}, E$, open, C, v_{targets}) (verifier)

Require: Coeffs $C \in \mathbb{F}_q^{m \times r}$, public prefixes $v_{\text{targets}} \in (\mathbb{F}_q^{n_{\text{cols}}})^m$.
1: Split open into mask openings for $[n_{\text{cols}}, n_{\text{cols}} + \ell)$ and tail openings at E.
2: Verify the DECS openings (degree soundness and Merkle authentication); reject on failure.
3: **for** each $k < m$ and each masked index $i < \ell$ **do**
4: Check $\sum_{j=0}^{r-1} C[k, j] \cdot \text{maskOpen.Pvals}[i][j] \stackrel{?}{=} \bar{v}_k[i] \pmod q$.
5: Reconstruct each batched polynomial $Q_k(X)$ from its Ω-prefix $v_{\text{targets}}[k]$ and mask tail $\bar{v}_k$.
6: Check tail consistency of Q_k against the opened linear combinations at points in E.
7: **return** accept.

C.4 LVCS and PCS/PIOP

LVCS provides linear opening queries to committed rows and is used to implement the polynomial oracle interface needed by PIOPs. SmallWood combines LVCS with DECS (for degree enforcement) to obtain a hash-based PCS.

C.5 What DECS/LVCS/PCS bind in ARC-SPRUCE

At a high level, the SmallWood stack provides a single commitment that simultaneously binds *all* witness rows used in a statement, and it enforces that every opened value is consistent with a low-degree polynomial view of those rows. In ARC-SPRUCE this binding is used in three complementary ways:

- **Row binding and low degree (DECS).** DECS ensures that the prover cannot answer openings using high-degree "fake" vectors: each committed row must correspond to a degree-$\le s + \ell - 1$ polynomial that matches the packed values on Ω and the masked tails on Ω'.
- **Linear linking across subsystems (LVCS).** LVCS is used to request openings of verifier-chosen linear combinations of rows. This is what couples different parts of the statement: the same opened evaluations are reused to recompute commitment residuals, centering residuals, the rational-hash residual, the signature residual, and the PRF checkpoint residuals.
- **No witness splitting (single PCS commitment).** Because all rows live under one commitment, the prover cannot use inconsistent witnesses for different checks. For instance, in showings the same signed M_2 row that appears in the hash/signature relation is also the row that key-binding constraints connect to the committed PRF key lanes; this prevents "mix-and-match" of (m_2, u) with an unrelated PRF witness.

This is exactly the binding invariant emphasized in the previous paper's SmallWood appendix: the verifier's checks are expressed as residual functions of opened row evaluations, so all constraints are evaluated against a single coherent row assignment.

C.6 Openings we request in issuance vs showing

Both proofs in ARC-SPRUCE follow the same pattern: the prover commits to all witness rows as polynomials, the verifier derives Fiat–Shamir challenges, and the prover opens the committed rows at verifier-chosen points. The verifier then recomputes residuals for each constraint family from these openings and checks that all residuals vanish.

Issuance (pre-sign). The witness consists of the base rows $M_1, M_2, R_0^U, R_1^U, \bar{R}, R_0, R_1$ while the target T and issuer challenges (r_0^I, r_1^I) are public. The verifier requests openings sufficient to replay: (i) the linear commitment constraints linking com to $(M_1, M_2, R_0^U, R_1^U, \bar{R})$, (ii) the centering constraints linking (R_0, R_1) to the holder/issuer randomness and carries (K_0, K_1), and (iii) the cleared-denominator hash constraint linking T to (M_1, M_2, R_0, R_1), together with packing and membership checks. Because all of these residuals are evaluated from the same opened rows, the proof binds the target T to the commitment opening and the centered randomness.

Showing (post-sign). The witness extends issuance with the hash target T, the signature preimage rows U, and the PRF witness block (key lanes and checkpointed S-box outputs). The verifier requests openings sufficient to replay: (i) the signature residual $A \cdot U - T$, (ii) the bilinear cleared-denominator hash residual linking T to the base rows, and (iii) the PRF key-binding, checkpoint, and tag-binding residuals (Section 5.5). The key point is that the PRF key lanes are tied back to the signed M_2 row via selector constraints, so the

opened values jointly bind tag = $\mathsf{F}(m_2, \text{nonce})$ to the same m_2 that appears in the signature relation.

Internal Ω-sum check. Even when an application's aggregated family is empty ($F_{\text{agg}} = \emptyset$), PACS/PIOP still enforces the masked Ω-sum test on the batch polynomials Q, which is the mechanism that ties the verifier's spot-check openings to correctness on the full packing domain Ω.

C.7 PACS/PIOP interface (used by our statements)

SmallWood's PACS/PIOP layer checks that constraint polynomials vanish on Ω (and at random points outside Ω), by committing to witness rows and mask polynomials and then opening them at verifier-chosen coordinates. The following algorithms summarize the high-level structure used in our proofs.

Algorithm 13 $\mathsf{PACS.PIOPCommitOracle}_{\text{Prover}}(\text{params}) \to \text{pcsCom}$

1: Interpolate witness rows into $P_1, \ldots, P_n \in \mathbb{F}_q[X]_{\le s+\ell'-1}$ with ℓ' random blind evaluations outside Ω.
2: Sample mask polynomials $M_1, \ldots, M_\rho \in \mathbb{F}_q[X]_{\le d_Q}$ such that $\sum_{\omega\in\Omega} M_i(\omega) = 0$ for all i.
3: Commit to $P \| M$ via PCS (implemented through LVCS/DECS); obtain pcsCom.
4: **return** pcsCom.

Algorithm 14 $\mathsf{PACS.PIOPBatchCombine}_{\text{Prover}}(\text{pcsCom}, \Gamma') \to Q$

1: Using P, M, form batched polynomials

$$Q_i(X) \;=\; M_i(X) + \sum_j \Gamma'_{i,j}(X)\, F_j(X) + \sum_u \gamma'_{i,u}\, F'_u(X),$$

where F_j and F'_u are the parallel and aggregated constraint polynomials evaluated on P.
2: **return** $Q = (Q_1, \ldots, Q_\rho)$.

Algorithm 15 PACS.PIOPOpenAndCheck(pcsCom, Q) (verifier)

1: Sample a query set $E' \subset S$ with $|E'| = \ell'$ (Fiat–Shamir in NIZK).
2: Verify PCS openings of P and M at E' (via LVCS/DECS); reject on failure.
3: Check that each $Q_i(e)$ matches its definition from $M_i(e)$ and the constraint evaluations at $e \in E'$.
4: Check the masked Ω-sum test $\sum_{\omega\in\Omega} Q_i(\omega) = 0$ for all i.
5: **return** accept/reject.

C.8 Fiat–Shamir compilation

SmallWood compiles the interactive protocol to a NIZK using Fiat–Shamir with grinding to obtain challenges with all-zero prefixes, as in [FR25]. Our ARC instantiation is dominated by parallel constraints, but the PACS/PIOP reminder here is stated for general $(F_{\text{par}}, F_{\text{agg}})$ statements. In particular, if the application introduces global coupling across the packing domain (as in the proof-friendly encodings inherited from the previous paper), those relations appear as F_{agg} and are enforced through the Ω-sum checks of the PIOP.

Algorithm 16 PACS.ToNIZK (Fiat–Shamir wrapper with grinding)

1: Sample salt $\xleftarrow{\$} \{0,1\}^{2\lambda}$.
2: Derive challenges by hashing the transcript with grinding counters to obtain required prefix properties.
3: Output PCS/LVCS/DECS openings and any required batched values; the verifier recomputes challenges and checks.

D Extended Parameters and Benchmarks

This appendix provides extended parameter discussion and additional benchmark data.

D.1 Admissibility checklist

A parameter tuple is *admissible* for ARC-SPRUCE if it simultaneously satisfies:

(1) **Bound vs. modulus:** $\beta_{\mathtt{msg}} \ll q$ so membership checks on $[-\beta_{\mathtt{msg}}, \beta_{\mathtt{msg}}]$ do not wrap modulo q.
(2) **Sampler hiding and shortness:** Gaussian widths exceed the required smoothing threshold and the signature bound $\beta_{\mathtt{sig}}$ holds with overwhelming probability.
(3) **Rational-hash guard:** denominator invertibility succeeds with negligible failure probability under the holder randomness distribution.
(4) **SmallWood degree/soundness:** the DECS/LVCS degree bounds and PIOP soundness parameters meet the target security λ.
(5) **PRF entropy:** truncation length ℓ_{tag} yields at least λ bits of tag entropy.

D.2 Smoothing and sampler budgets

Let $\eta_\varepsilon(\Lambda)$ denote the smoothing parameter of a lattice Λ. A sufficient condition for statistical hiding in Gaussian preimage sampling is to choose Gaussian widths above a floor $\sigma_{\min} \ge \eta_\varepsilon(\Lambda_\perp(A))$, with slack (Appendix B). For the ambient lattice $\mathbb{Z}^{2N}$ one may use the classical bound

$$\eta_\varepsilon(\mathbb{Z}^{2N}) \;=\; \sqrt{\ln\big(4N(1+2^\lambda)\big)/\pi}.$$

In our tuning, trapdoor quality α and acceptance parameters are chosen so that the output shortness bound $\beta_{\mathtt{sig}}$ holds with overwhelming probability.

D.3 Example lattice parameters

Table 2 records one concrete instantiation used for the proof sizes in Table 1.

Knob / Result	Symbol	Value (example)
Ring degree	N	1024
Modulus	q	1038337
Security target	λ	128
Trapdoor quality	α	≈ 1.23 (measured)
Gaussian width scale	σ	tuned per-slot (Appendix B)
Signature bound	$\beta_{\mathtt{sig}}$	derived from sampler tail bound

Table 2: Example lattice parameters (illustrative; see Appendix B for the sampling interface and tuning).

D.4 SmallWood soundness terms

SmallWood's soundness is a combination of degree soundness (DECS), LVCS binding, and PIOP sampling errors [FR25]. For parameters $(N, s, \ell, \ell', \rho, \eta)$ and degree bounds $d_{\text{decs}} = s + \ell - 1$ and d_Q, typical upper bounds take the form:

$$\varepsilon_{\text{decs}} \lesssim \binom{N}{d_{\text{decs}}+2} q^{-\eta}, \qquad \varepsilon_{\text{sum}} = q^{-\rho}, \qquad \varepsilon_{\text{tail}} \approx \frac{\binom{d_Q}{\ell'}}{\binom{|S|}{\ell'}}.$$

Our issuance statement uses only linear constraints (given public T), so d_Q is driven by membership checks; the showing statement additionally depends on the PRF effective degree (Appendix E).

D.5 SmallWood knobs (example)

For the issuance and showing proofs reported in Table 1, one example parameter tuple is:

$$(s, \ell, \ell', \rho, \theta, \eta) \;=\; (16, 25, 2, 2, 4, 19),$$

with masking degree d_Q derived from the instantiated constraint degrees. The pre-sign statement remains linear in the witness (given public T), while the post-sign statement's effective degree is dominated by the PRF S-box composition.

D.6 Extended benchmark grid (SmallWood)

For additional context on how SmallWood knobs affect proof size and proving time, Table 3 reports an example sweep over packing and soundness parameters (small-field variant).

ProofKB	Time (s)	Bits	NCols	ℓ	ℓ'	ρ	θ	η
9.32	0.266	133.44	6	22	2	2	4	17
10.07	0.228	146.93	4	24	2	2	4	17
11.52	0.250	133.22	4	28	2	2	4	17
14.06	0.630	198.01	4	34	8	5	2	26
14.17	0.654	192.26	6	34	8	5	2	26
14.24	0.670	198.48	4	34	8	6	2	26

Table 3: Example sweep over SmallWood knobs (proof size, prover time, and soundness bits). Sizes exclude public parameters.

E PRF Details and Miscellaneous

This appendix records a concrete ZK-friendly PRF instantiation compatible with the canonical ARC protocol.

E.1 PRF specification

We use a Poseidon2-like permutation with feed-forward and truncation. Let $k \in \mathbb{F}_q^{\ell_{\text{key}}}$ be the secret key (derived from the signed $\mathbf{k}$) and $n \in \mathbb{F}_q^{\ell_{\text{nonce}}}$ encode the public nonce. Let $t = \ell_{\text{key}} + \ell_{\text{nonce}}$ be the permutation width. Define

$$\mathsf{F}(k, n) \;=\; \mathrm{Tr}\big(P(k\|n) + (k\|n)\big),$$

where $\mathrm{Tr} : \mathbb{F}_q^t \to \mathbb{F}_q^{\ell_{\text{tag}}}$ outputs the first ℓ_{tag} lanes and P is a Poseidon2-style permutation.

E.2 Permutation structure

Let $d \geq 3$ be the smallest constant such that $\gcd(d, q-1) = 1$. Fix round counts R_F (external rounds, even) and R_P (internal rounds), round constants, and MDS matrices M_E, M_I. The permutation has the form

$$P \;=\; E_{R_F} \circ \cdots \circ E_{R_F/2+1} \circ I_{R_P} \circ \cdots \circ I_1 \circ E_{R_F/2} \circ \cdots \circ E_1.$$

Each external round applies the S-box to all lanes, while each internal round applies the S-box to lane 1 only. This structure reduces nonlinear constraints in the post-sign NIZK while retaining diffusion.

External and internal rounds. Let $x = (x_1, \ldots, x_t) \in \mathbb{F}_q^t$. For each external round $i \in [1, R_F]$, fix per-lane constants $(c_1^{(i)}, \ldots, c_t^{(i)}) \in \mathbb{F}_q^t$. For each internal round $i \in [1, R_P]$, fix a scalar constant $\hat{c}^{(i)} \in \mathbb{F}_q$. Then:

$$E_i(x) \;=\; M_E \cdot \big((x_1 + c_1^{(i)})^d, \ldots, (x_t + c_t^{(i)})^d\big)^\top,$$

$$I_i(x) \;=\; M_I \cdot \big((x_1 + \hat{c}^{(i)})^d,\; x_2, \ldots, x_t\big)^\top.$$

The matrices $M_E, M_I \in \mathbb{F}_q^{t \times t}$ are chosen so that they are invertible and provide sufficient diffusion (e.g., Cauchy-style MDS matrices as in Poseidon2).

E.3 Parameter generation

All PRF parameters are public. A typical parameter generator fixes:

- the base field modulus q (shared with the rest of the protocol),
- the S-box exponent d with $\gcd(d, q-1) = 1$,
- width $t = \ell_{\text{key}} + \ell_{\text{nonce}}$ and truncation ℓ_{tag},
- round counts R_F, R_P,
- MDS matrices M_E, M_I and round constants $\{c^{(i)}\}, \{\hat{c}^{(i)}\}$.

The truncation length is chosen to satisfy $\ell_{\text{tag}} \geq \lceil \lambda / \log_2(q) \rceil$, ensuring that the tag contains λ bits of entropy.

E.4 Arithmetization strategy (System A with checkpoints)

To keep showing proofs compact, we commit only selected nonlinear checkpoints:

- **Key lanes.** We commit one row per key lane $\mathsf{Key}[i]$ and enforce equality to an encoding of signed $\mathbf{k}$ using selector constraints at fixed Ω-coordinates.
- **S-box outputs.** We commit only the S-box outputs at a checkpoint schedule (e.g., every internal round and periodically among external rounds). Linear wiring between checkpoints (add-round-constants, MDS multiplications) is recomputed by the verifier from opened evaluations and public constants.

This yields a System-A style constraint system where the dominant degree comes from composing at most a small number of unchecked S-box layers between checkpoints.